



Escuela Politécnica Superior

DS y ML

Machine Learning y Deep Learning

Graph Machine Learning

Aprendizaje Automático basado en Grafos

Álvaro Ortigosa, alvaro.ortigosa@uam.es



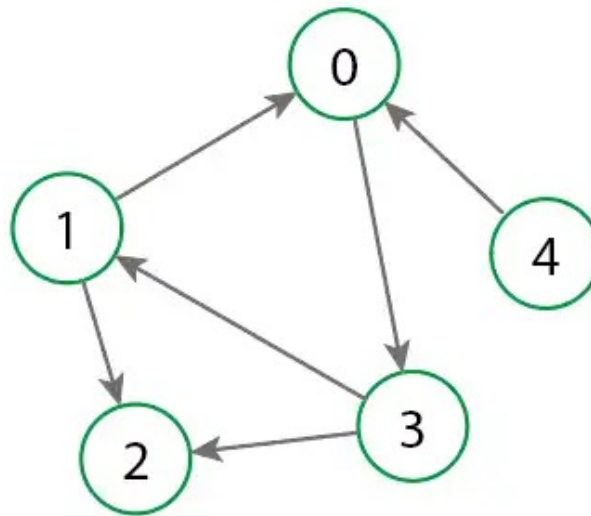
Aprendizaje automático basado en grafos - Introducción

Cuando las **relaciones** también tienen información

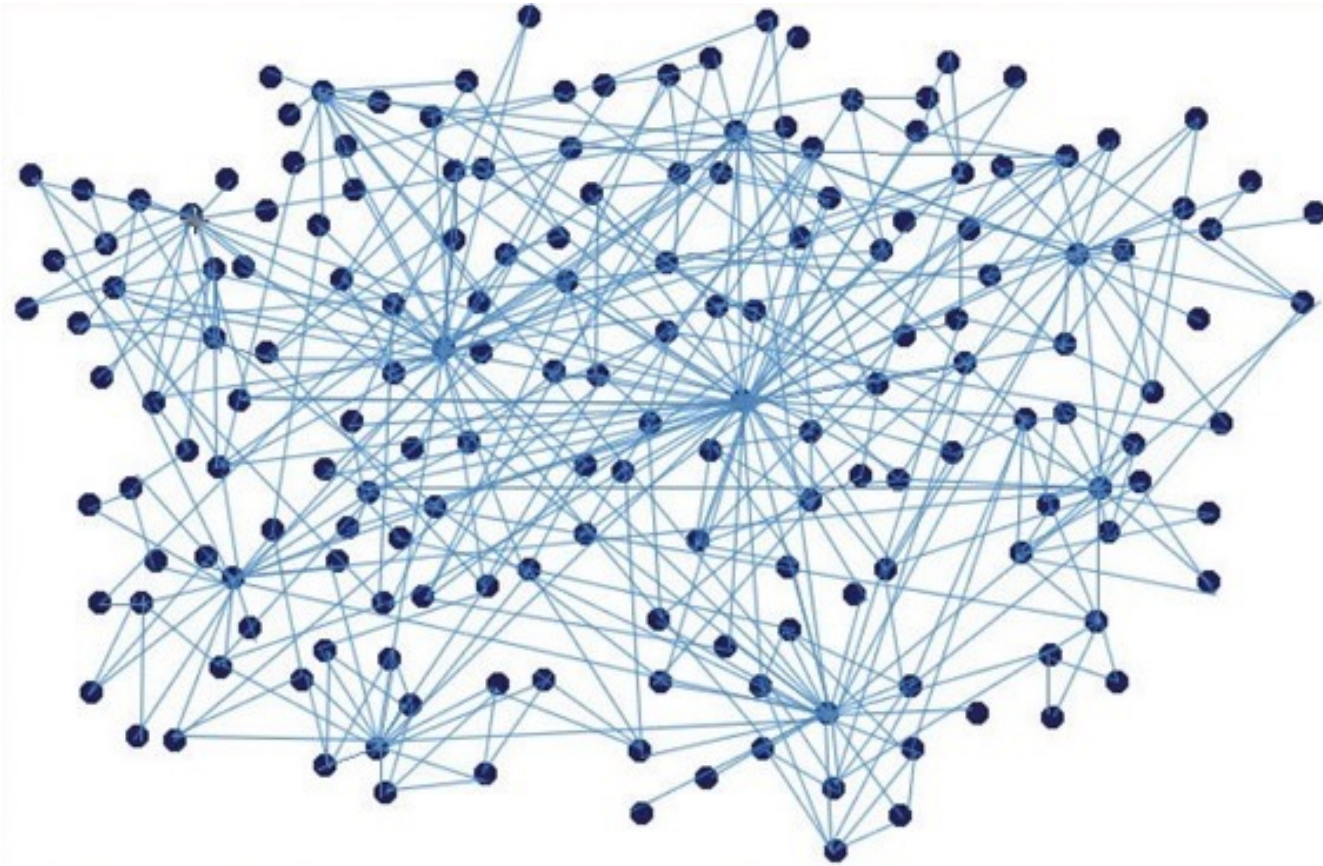
- Veremos una familia de técnicas distinta del ML tabular clásico.
- No vamos a hacer mucha matemática
- El objetivo es entender ideas, aplicaciones y límites



Empecemos por el principio... ¿qué es un grafo?

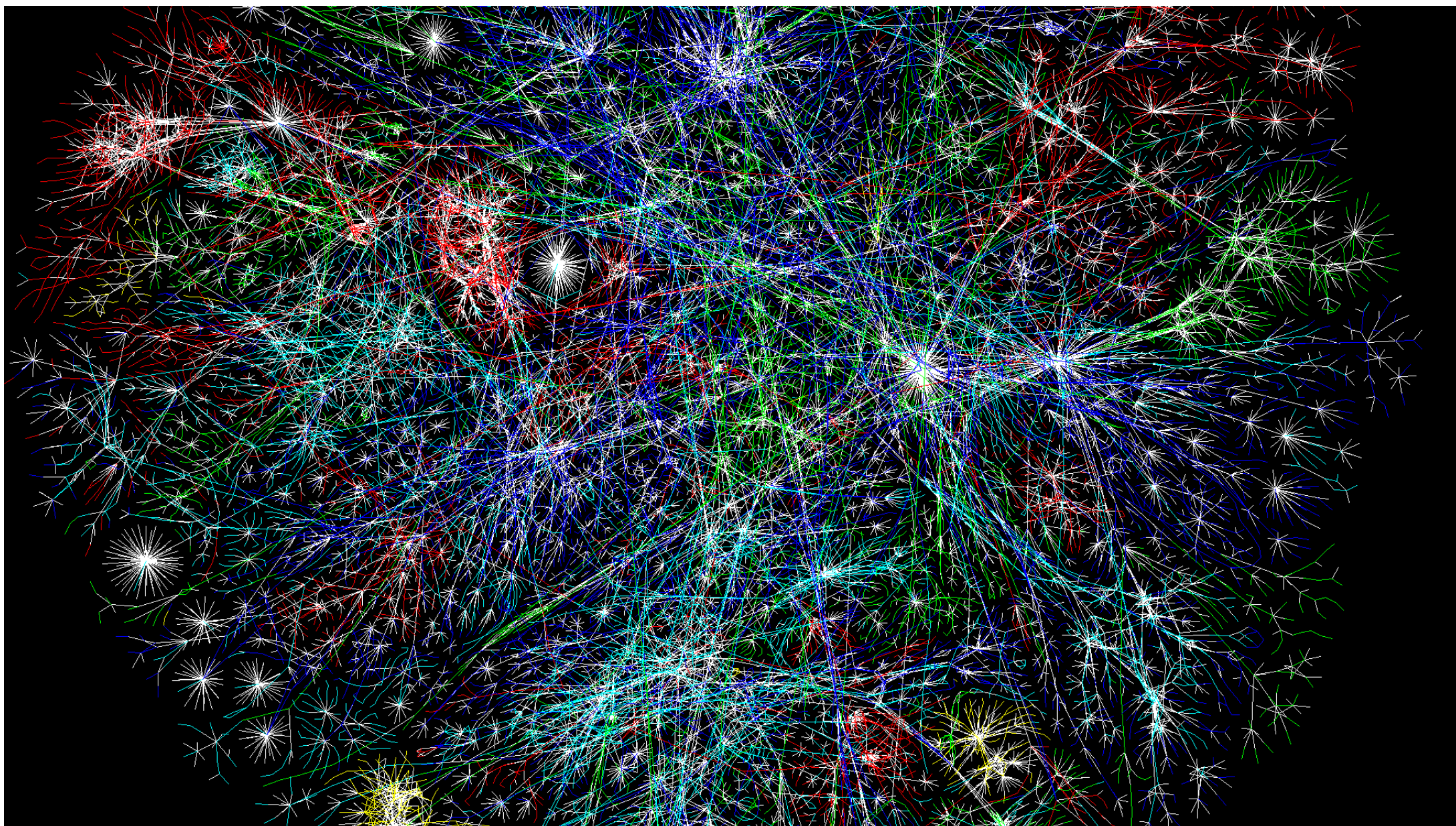


Empecemos por el principio... ¿qué es un grafo?



Empecemos por el principio... ¿qué es un grafo?





¿Qué formas conocemos de representar datos para DS y ML?

- Aunque ha permitido avanzar la Ciencia de Datos y la IA (casi) hasta lo que hoy conocemos, en realidad no todos los dominios que queremos modelar/representar “caben” en este modelo de tabla

Tabular Data

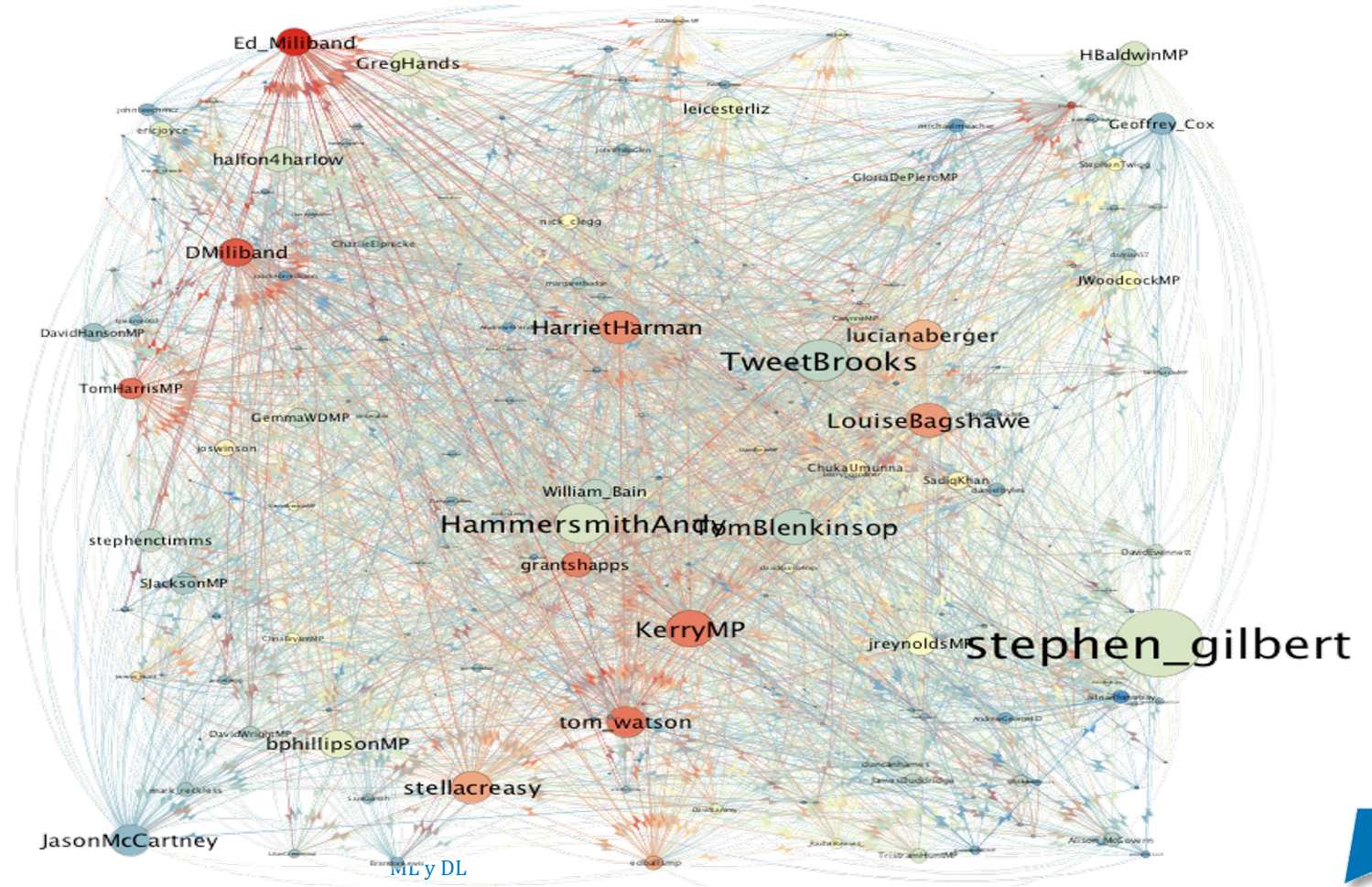
columns = attributes for those observations

Rows = observations

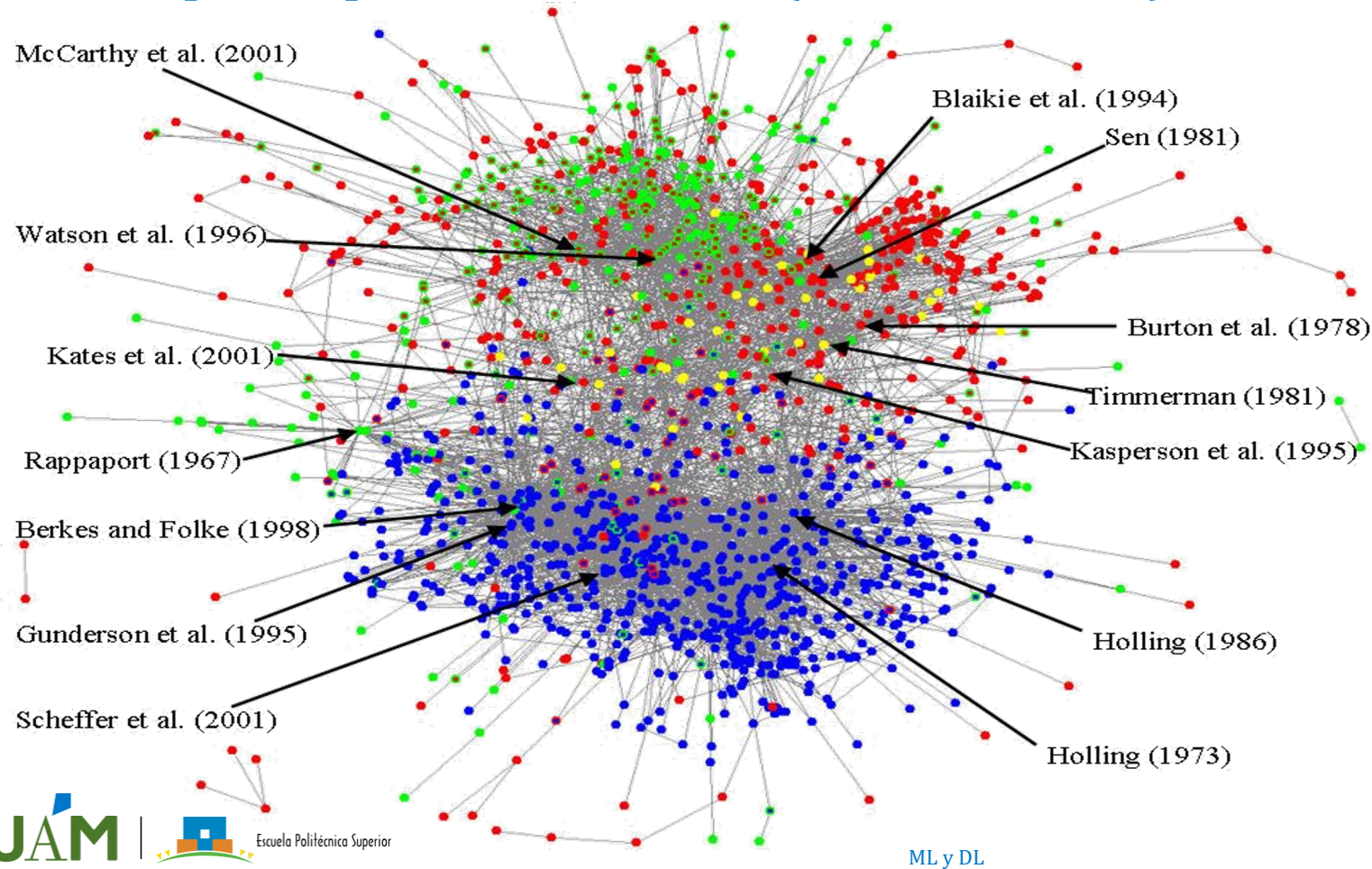
Player	Minutes	Points	Rebounds	Assists
A	41	20	6	5
B	30	29	7	6
C	22	7	7	2
D	26	3	3	9
E	20	19	8	0
F	9	6	14	14
G	14	22	8	3
I	22	36	0	9
J	34	8	1	3

Datos que se pueden modelar (naturalmente) como un grafo

Redes X (Twitter)

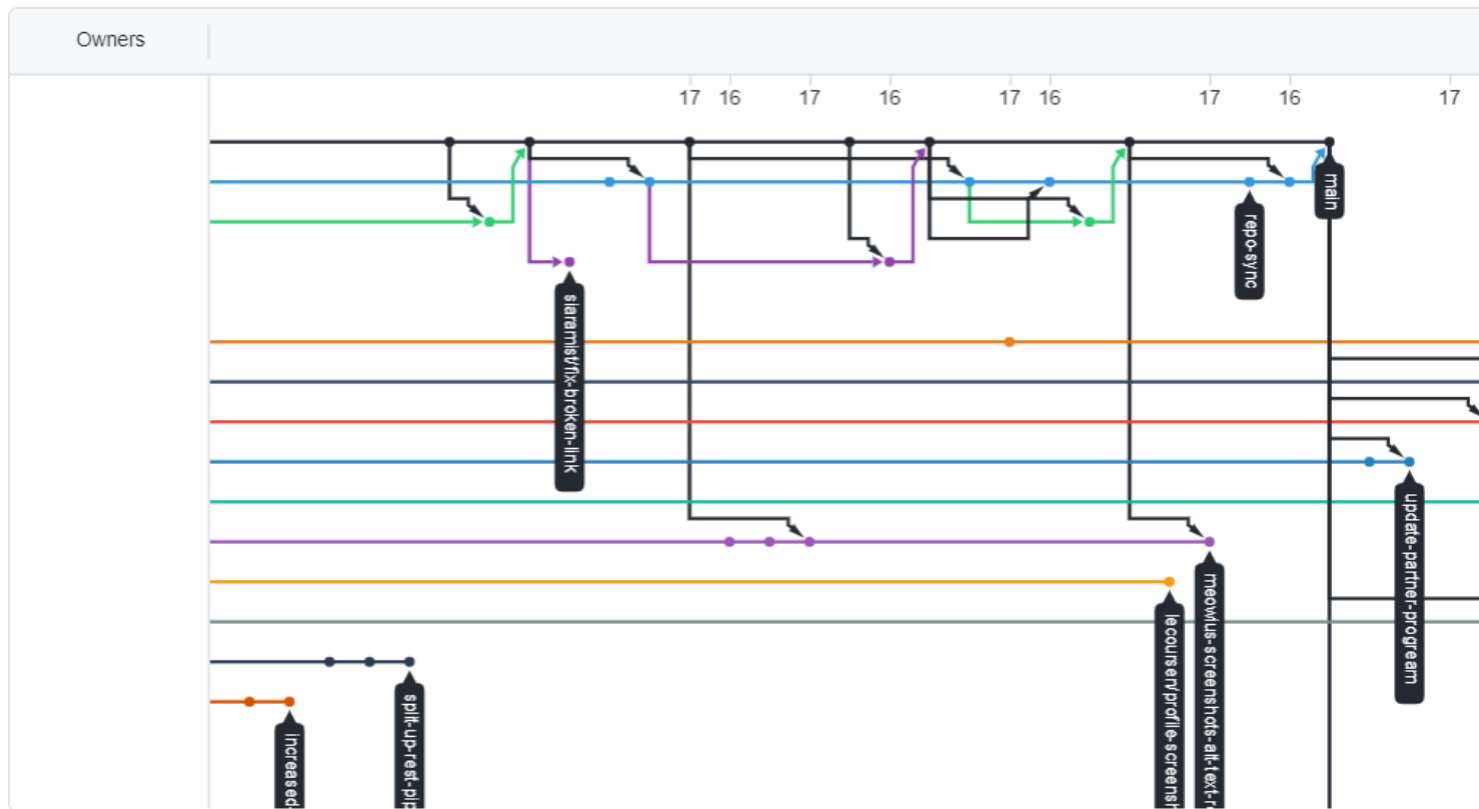


Datos que se pueden modelar (naturalmente) como un grafo



**Redes de citas
científicas**

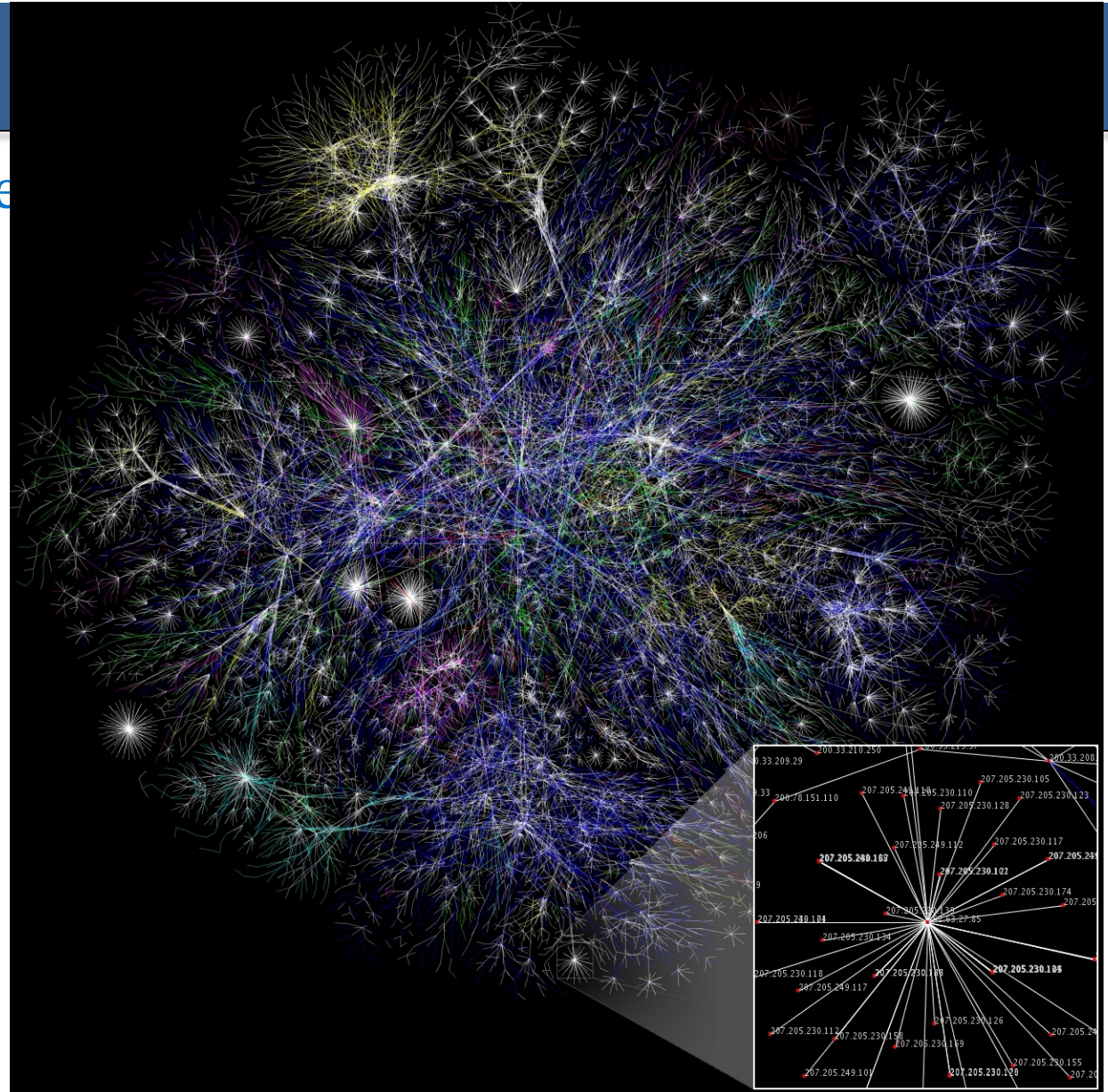
Datos que se pueden modelar (naturalmente) como un grafo



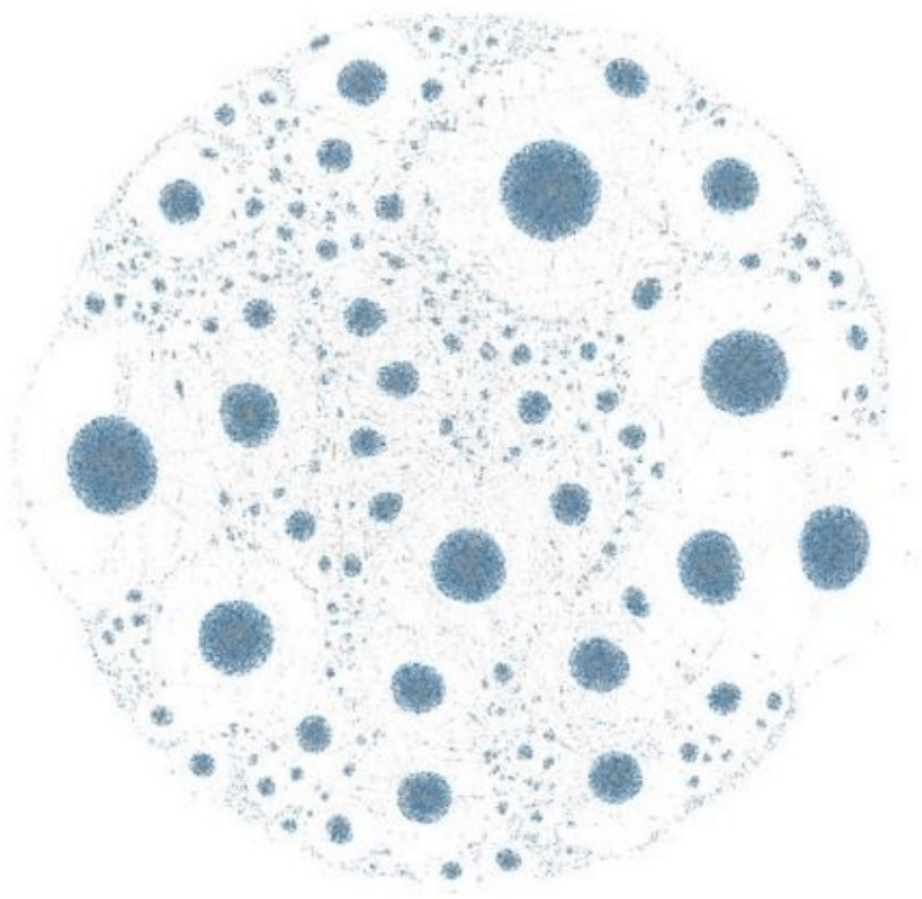
Versiones de software (Github)

Datos que se pueden modelar

Internet

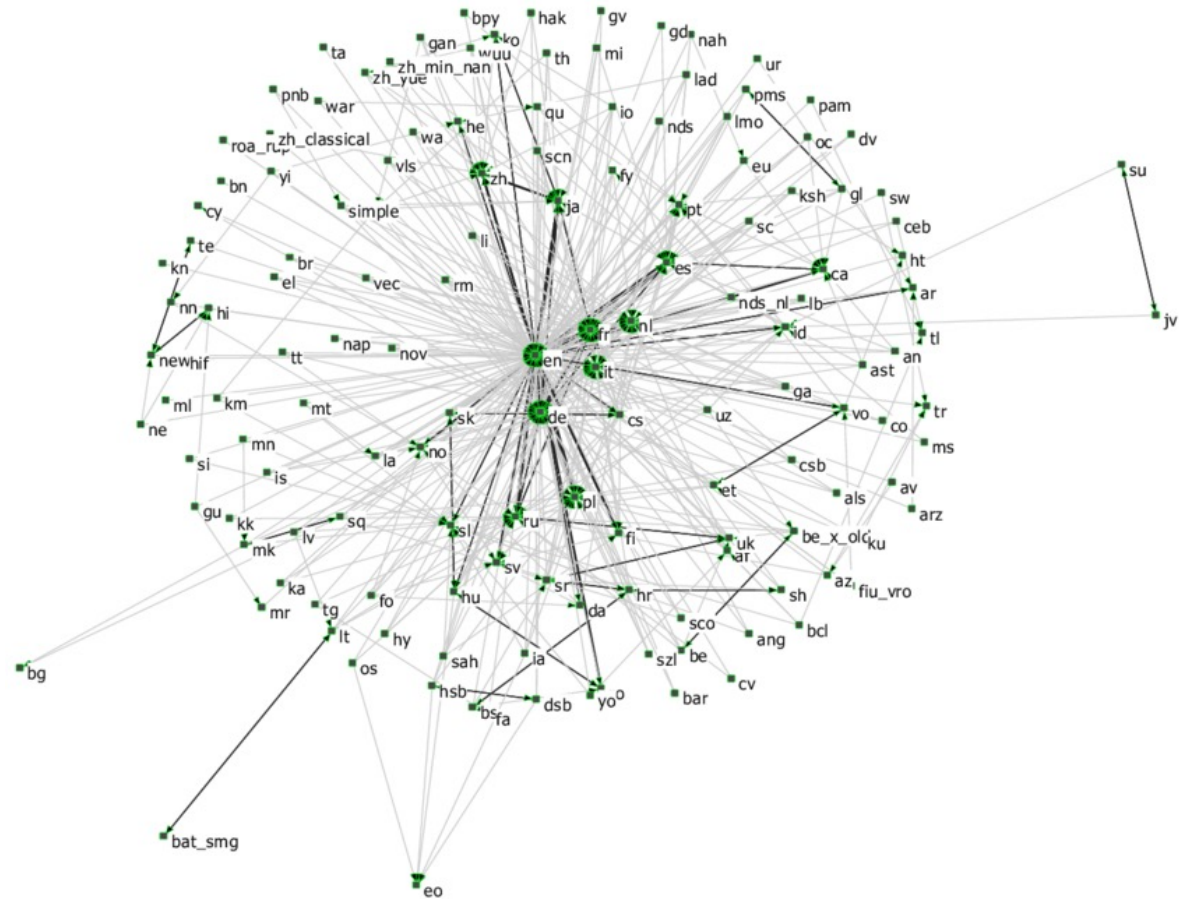


Datos que se pueden modelar (naturalmente) como un grafo



Sitios DarkWeb

Relaciones entre idiomas (Wikipedia)



Datos que se pueden modelar (naturalmente) como un grafo



Mapa carreteras

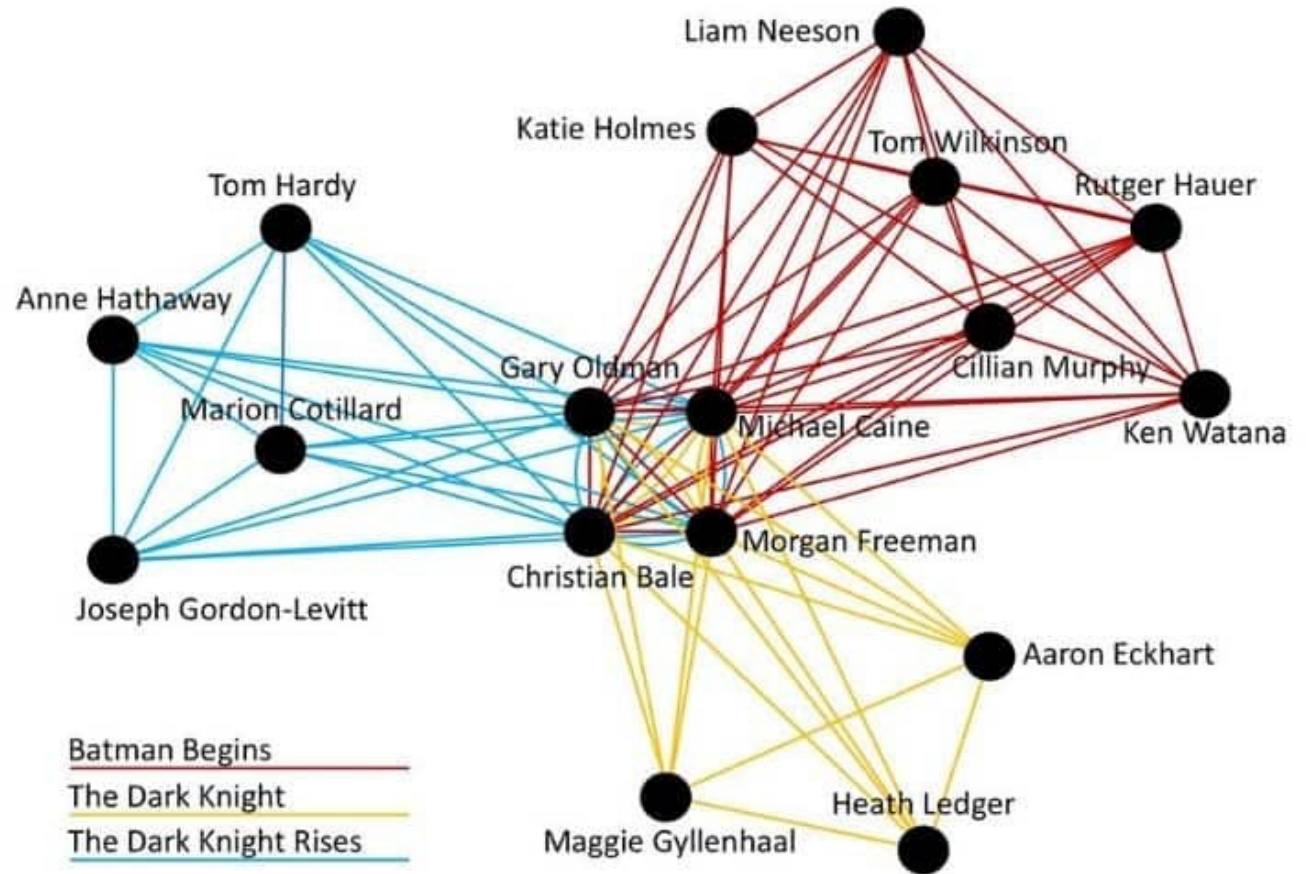
Datos que se pueden modelar (naturalmente) como un grafo

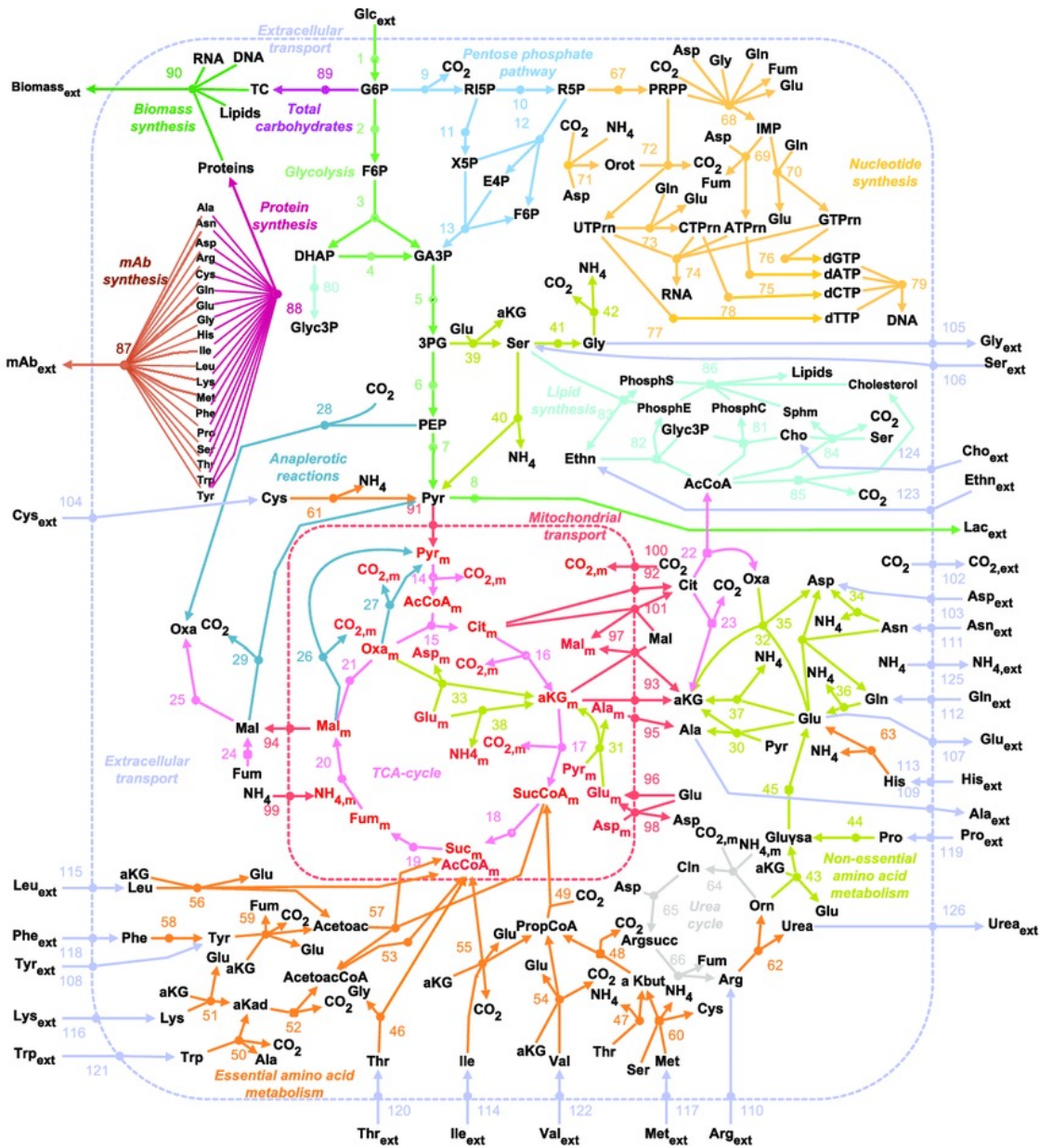


Plano metro

Datos que se pueden modelar (naturalmente) como un grafo

Actores y películas

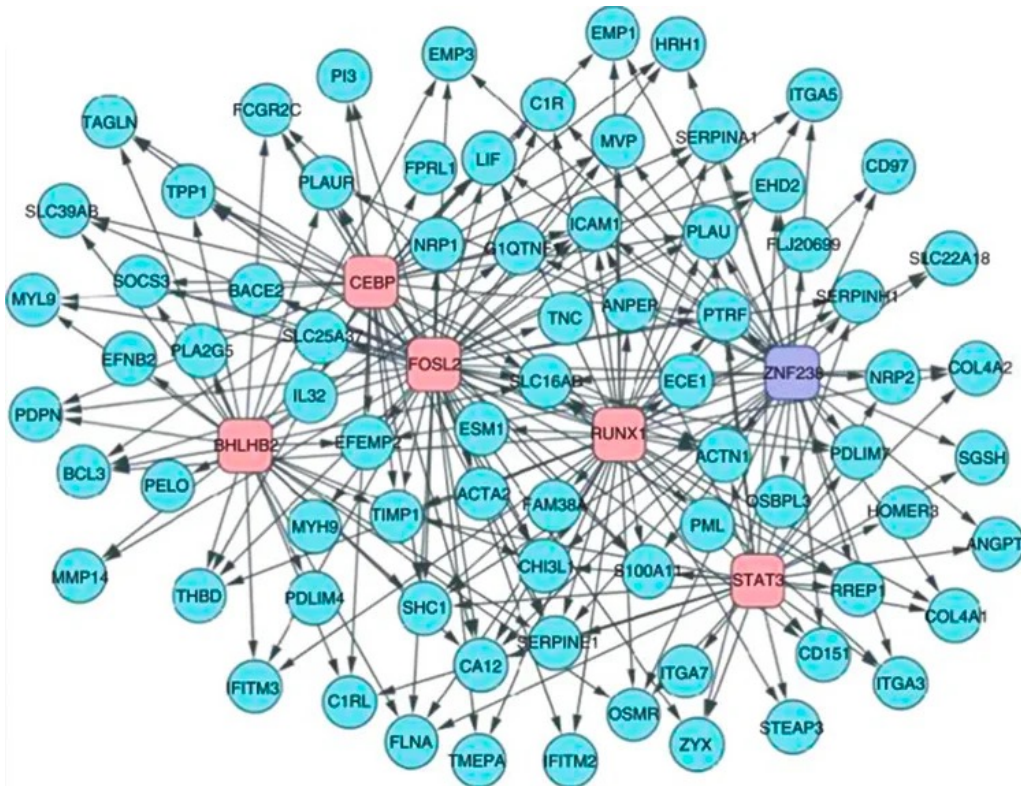




lmente) como un grafo

Redes metabólicas

Datos que se pueden modelar (naturalmente) como un grafo



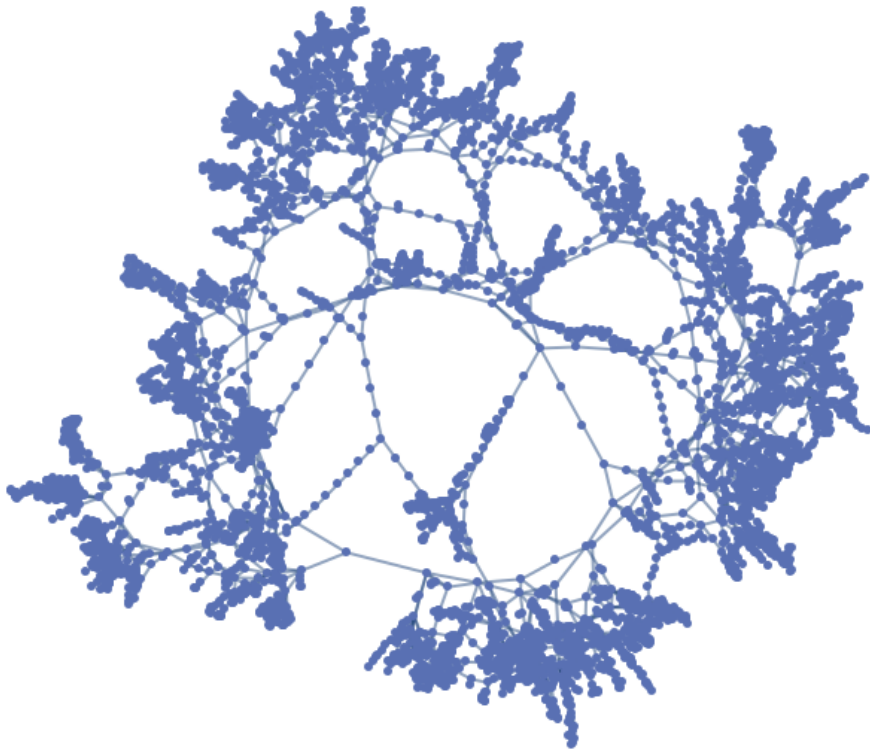
Redes Regulatorias de Células/Genes

Datos que se pueden modelar (naturales)

**Redes
neuronales
naturales**

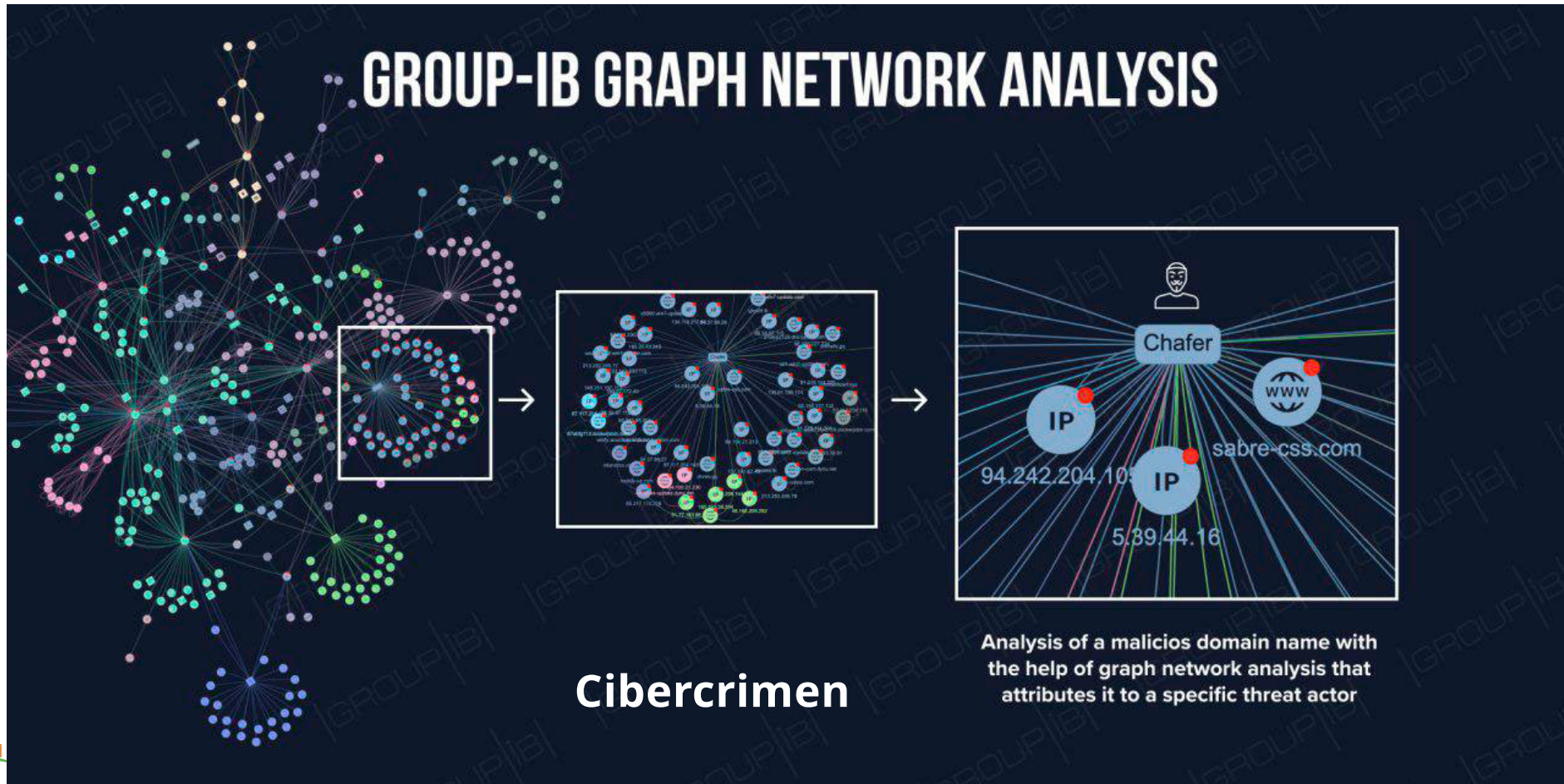


Datos que se pueden modelar (naturalmente) con

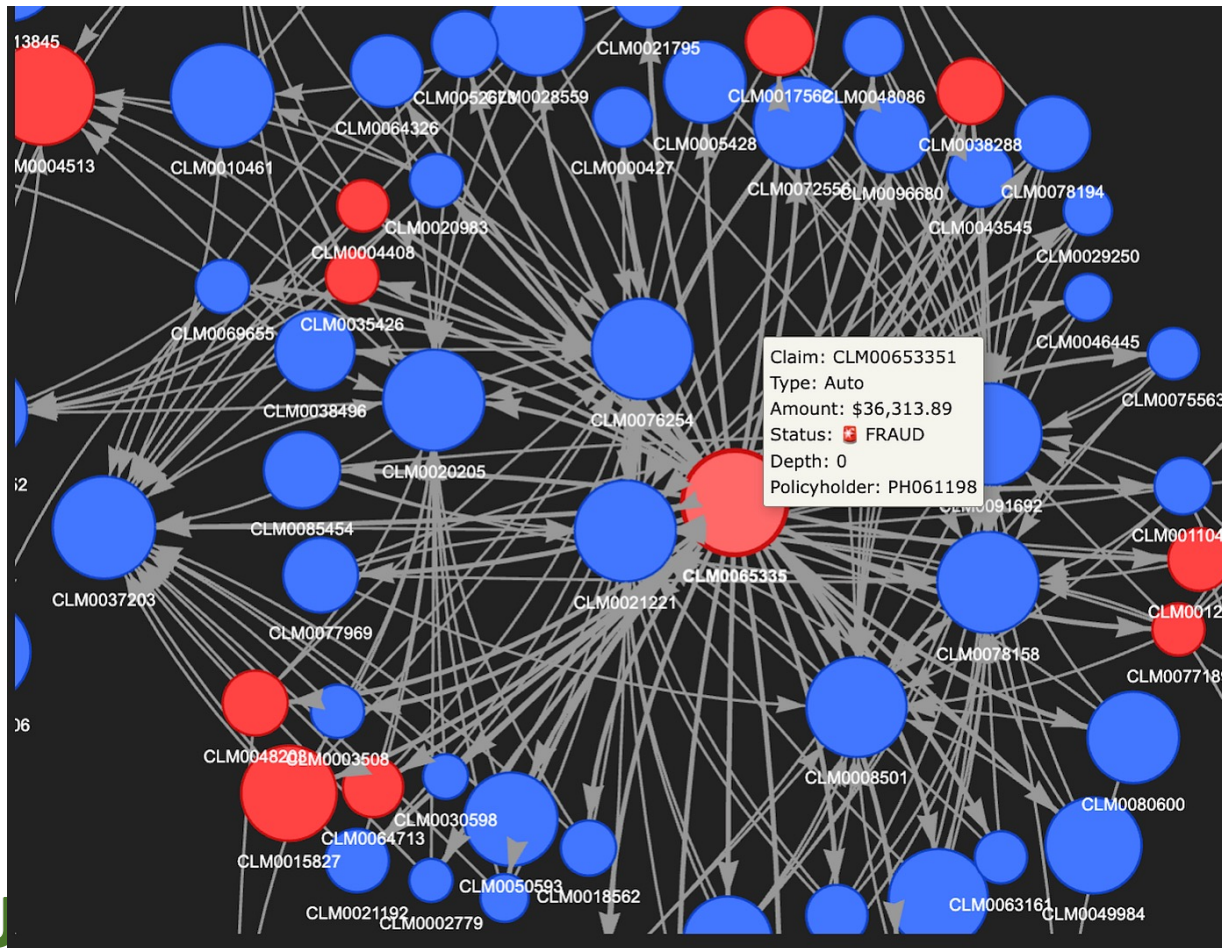


**Redes
eléctricas**

Datos que se pueden modelar (naturalmente) como un grafo



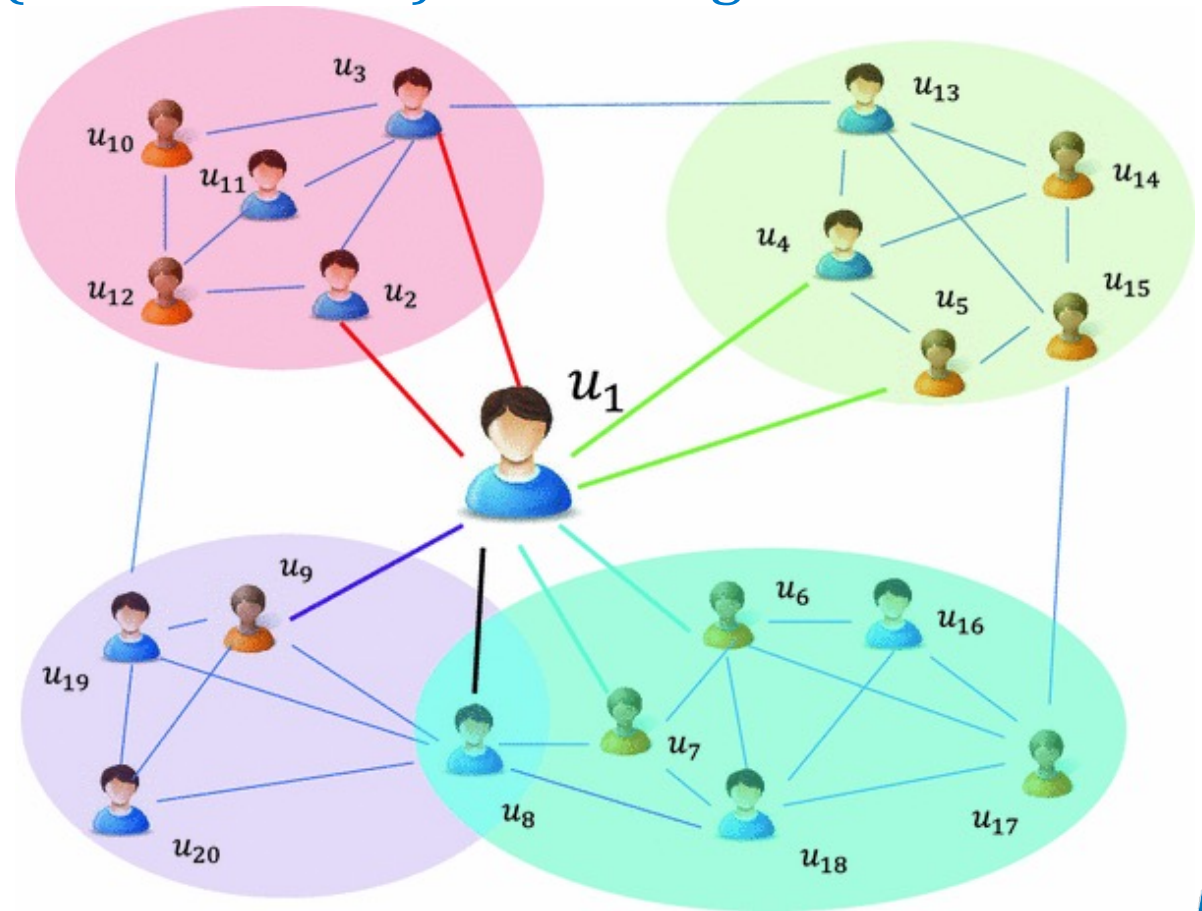
Datos que se pueden modelar (naturalmente) como un grafo



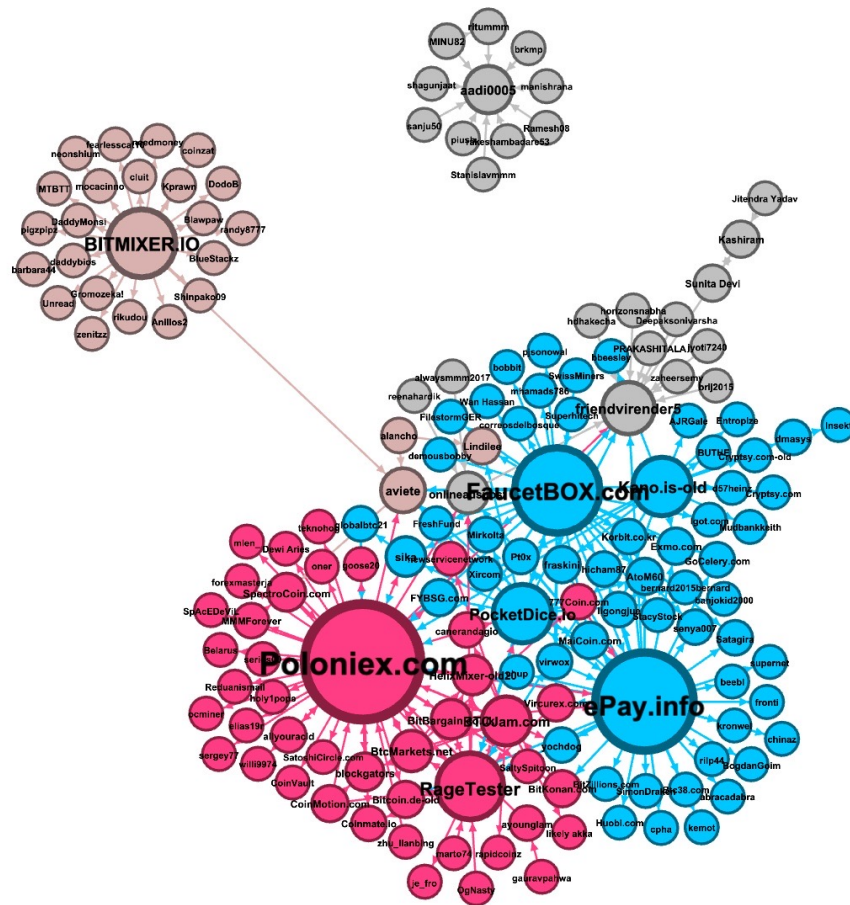
Fraude

Datos que se pueden modelar (naturalmente) como un grafo

Sistemas de Recomendación



Datos que se pueden modelar (naturalmente) como un grafo



Redes de criptomonedas

¿Por qué las tablas no son suficientes?

- En ML clásico:
 - Cada fila suele representar un ejemplo/ejemplar/instancia/observación.
 - Asumimos que cada ejemplo es más o menos independiente.
 - Eso funciona bien en muchos problemas, pero **no en todos**.
- Ejemplo donde no funciona
 - Dos usuarios con las mismas características.
 - Uno pertenece a una red de fraude y otro no.
 - Solo mirando la fila no los distingues bien.
- En muchos problemas, un objeto no se entiende bien solo por sus atributos, sino también por su contexto relacional → la **estructura de relaciones** añade información.

Contexto de uso

- ¿En qué problemas de la vida real importa “quién está conectado con quién”?
 - ¿Creéis que una tabla normal bastaría para representar una red social?
 - ¿Qué información perderíamos si ignoramos las conexiones?
-
- Un **grafo** es una representación de datos donde las **relaciones forman parte del problema**.

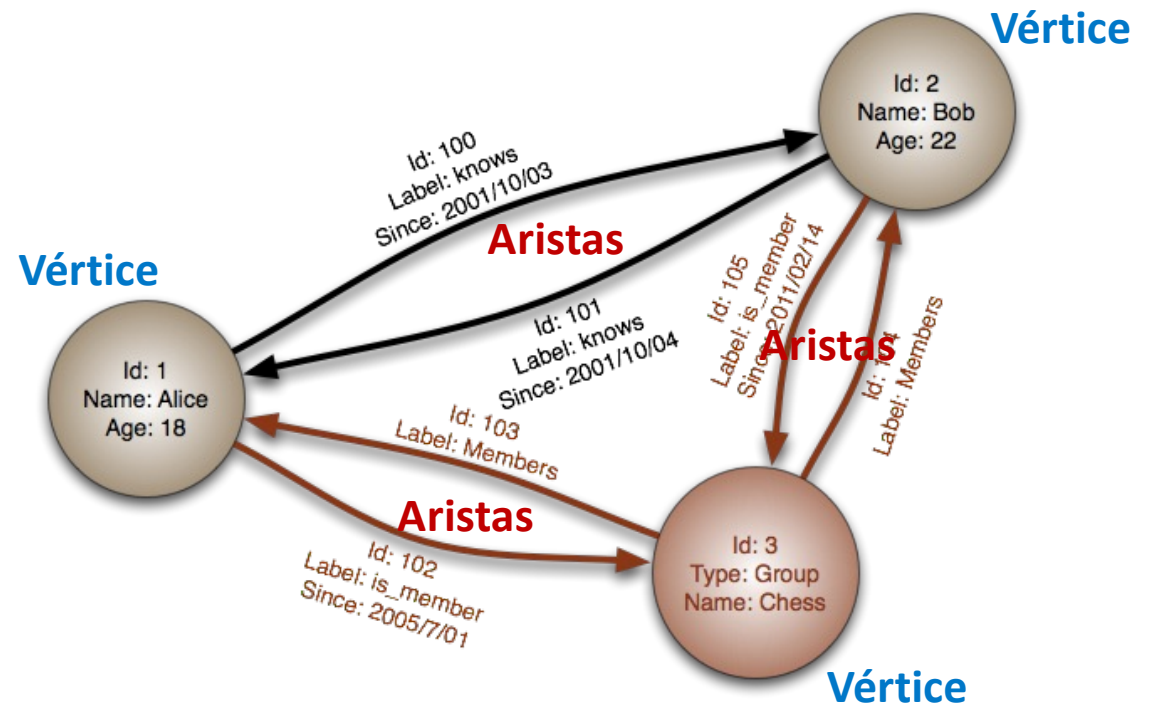


¿Qué es un grafo?

- Ejemplos:
 - persona $\leftrightarrow$ persona
 - ciudad $\leftrightarrow$ ciudad
 - ordenador $\leftrightarrow$ ordenador
 - átomo $\leftrightarrow$ átomo
- Pero también
 - película $\leftrightarrow$ actor
 - cartera $\leftrightarrow$ transacción
 - usuario $\leftrightarrow$ tweet

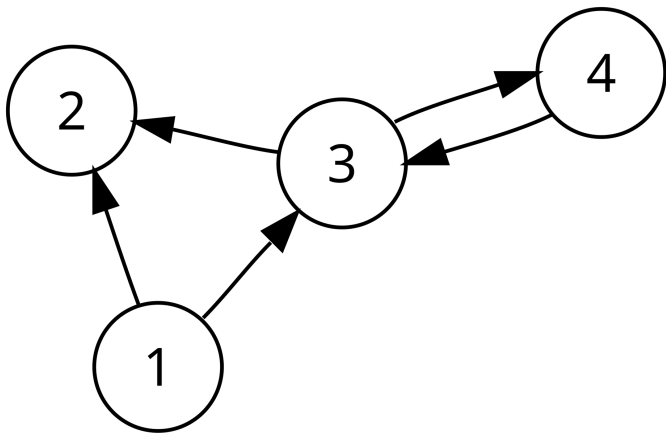
Nodos o vértices = entidades

Aristas = relaciones

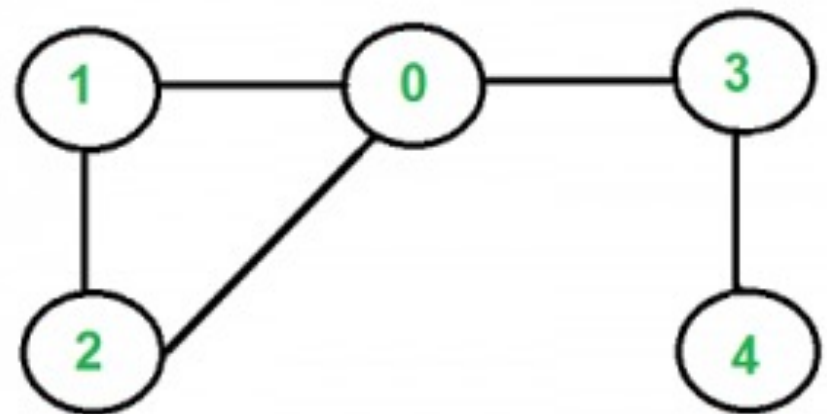


Tipos básicos de grafos

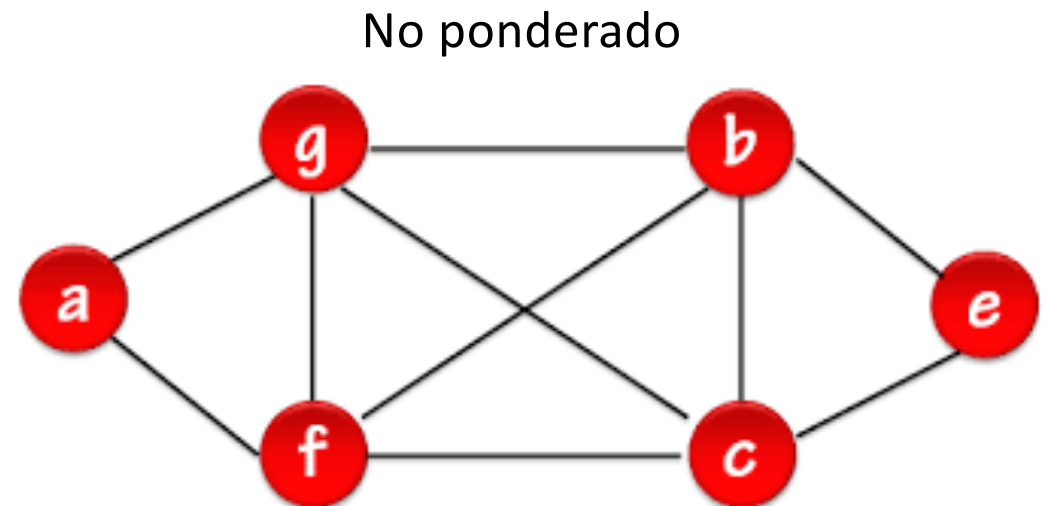
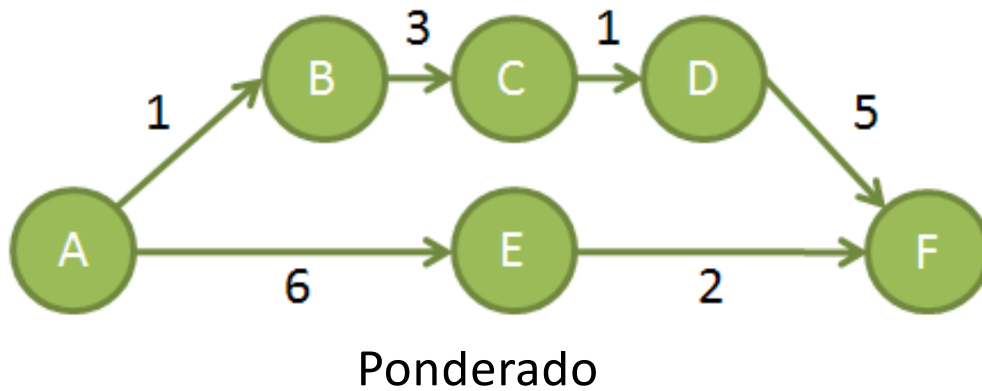
Dirigidos



No dirigidos

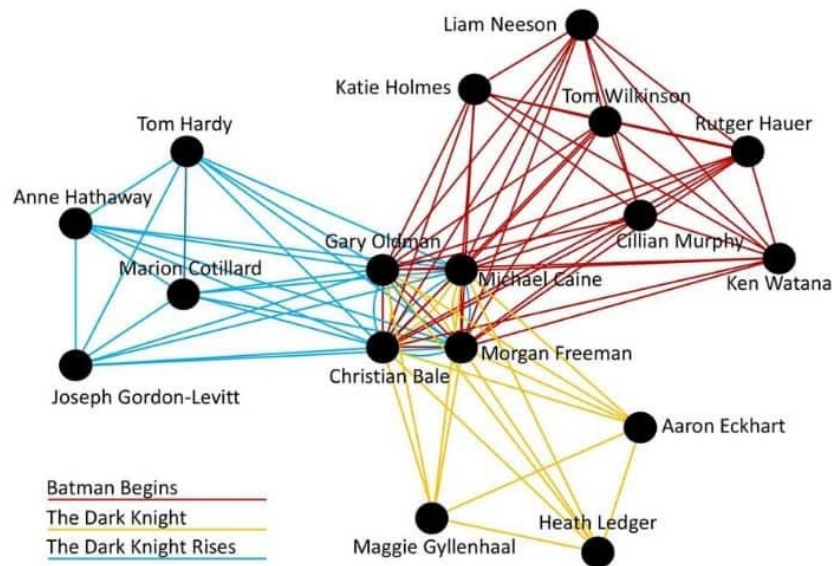


Tipos básicos de grafos



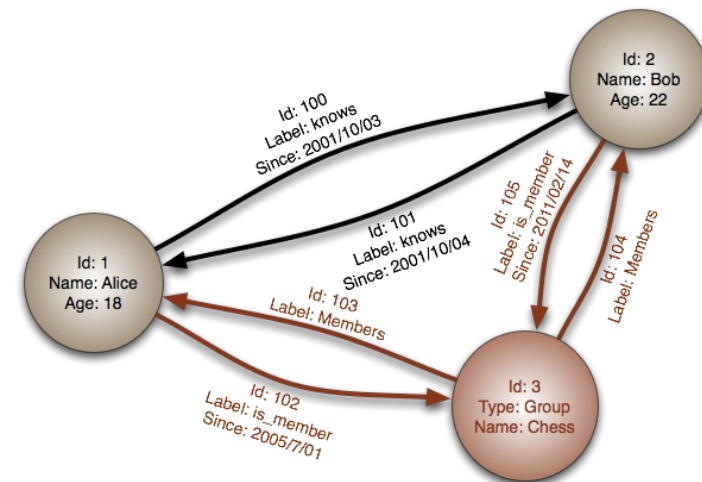
Tipos básicos de grafos

Homogéneo



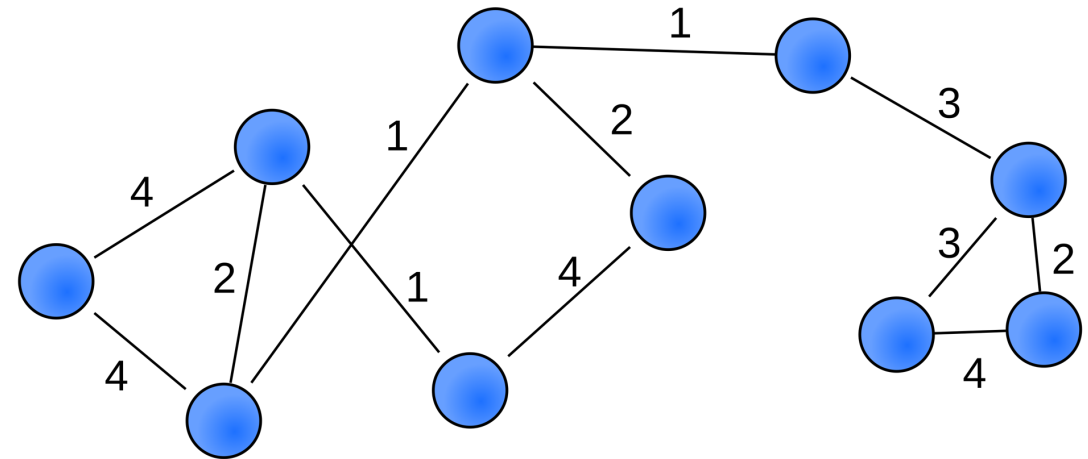
(todos los nodos representan el **mismo tipo** de entidad)

No homogéneo



Qué aporta representar algo como grafo

- Captura relaciones explícitas
- Incorpora contexto local
- Permite estudiar estructura global
- Ayuda a detectar patrones no visibles en tablas



Ahora vosotros

- ¿Tabla, secuencia, imagen o grafo?
 - clientes de un banco
 - transferencias entre cuentas bancarias
 - temperatura por día
 - ciudades conectadas por vuelos



Tareas típicas en aprendizaje sobre grafos

- Cuatro tareas principales:
 - Predicción sobre nodos
 - Predicción sobre aristas
 - Predicción sobre grafos completos
 - Detección de comunidades (*clustering*)



Predicción sobre nodos

- El objeto a predecir es un nodo concreto
- Ejemplos:
 - clasificar un perfil como bot o no bot.
 - detectar cuentas sospechosas.
 - clasificar documentos en una red de citas académicas
 - detectar transacciones fraudulentas.

Clasificación de nodos

- Ejemplo:
 - Supongamos que queremos identificar los perfiles de una red social gestionados por bots.
 - Nos gustaría un modelo que pueda clasificar autónomamente bots a partir de un número pequeño de ejemplos etiquetados.
- Objetivo: predecir la etiqueta y_u - tipo, categoría o atributo – asociada con todos los nodos $u \in V$, conociendo solo las verdaderas etiquetas de un conjunto de nodos $V_{\text{entrenamiento}} \subset V$.
 - Normalmente se conocen las etiquetas de unos pocos nodos de un solo grafo; a veces puede implicar la generalización a otros grafos no conectados.

Clasificación de nodos

- Los enfoques más exitosos para clasificación de nodos aprovechan precisamente las relaciones entre nodos:
 - Explotar la homofilia: los nodos comparten características con sus vecinos.
 - Equivalencia estructural: nodos con estructuras de vecinos locales similares tendrán etiquetas similares.
 - Heterofilia: asume que los nodos se conectan preferentemente con nodos con etiquetas distintas a la suya.

Clasificación de datos: ¿supervisado o no supervisado?

- Cuando se entrena un modelo para clasificar nodos, se tiene acceso a todo el grafo: algunos (pocos) nodos estarán etiquetados (conjunto entrenamiento), y otros no (conjunto test).
- Pero la información de los nodos de test (cómo se relacionan en el grafo) puede ser usada en el entrenamiento.
- Como el entrenamiento se hace con datos etiquetados y no etiquetados se denomina aprendizaje semi-supervisado.
 - Sin embargo, la formulación estándar del aprendizaje semi-supervisado asume la condición de independencia de los elementos, que en los grafos no se cumple.

Predicción sobre aristas

- El modelo estima si una relación existe, aparecerá o es relevante
- Ejemplos:
 - recomendar amistad
 - recomendar producto
 - predecir relación futura entre dos entidades
 - detectar conexión fraudulenta

Predicción de relaciones

- ¿Qué pasa si solo tenemos información parcial sobre las relaciones? ¿Podemos inferir las relaciones que nos faltan?
- Dependiendo del dominio lo llamamos predicción de enlaces, completado de grafo, inferencia o predicción de relaciones.
 - Por ejemplo, recomendar contenidos a los usuarios de una plataforma social.
- Objetivo: dado un conjunto de vértices V y un conjunto incompleto de aristas entre esos nodos $A_{\text{entrenamiento}} \subset A$, usar esta información parcial para inferir el resto de las aristas $A \setminus A_{\text{entrenamiento}}$.

Predicción de relaciones

- Igual que pasa con la clasificación de nodos, este tipo de tarea difumina los límites de entre las categorías tradicionales de ML: a menudo se la considera supervisada y no supervisada.
- Dependiendo del objetivo, las predicciones de relaciones se pueden hacer sobre un único grafo estático, sobre grafos dinámicos, e incluso sobre múltiples grafos disjuntos.

Predicción sobre grafos completos

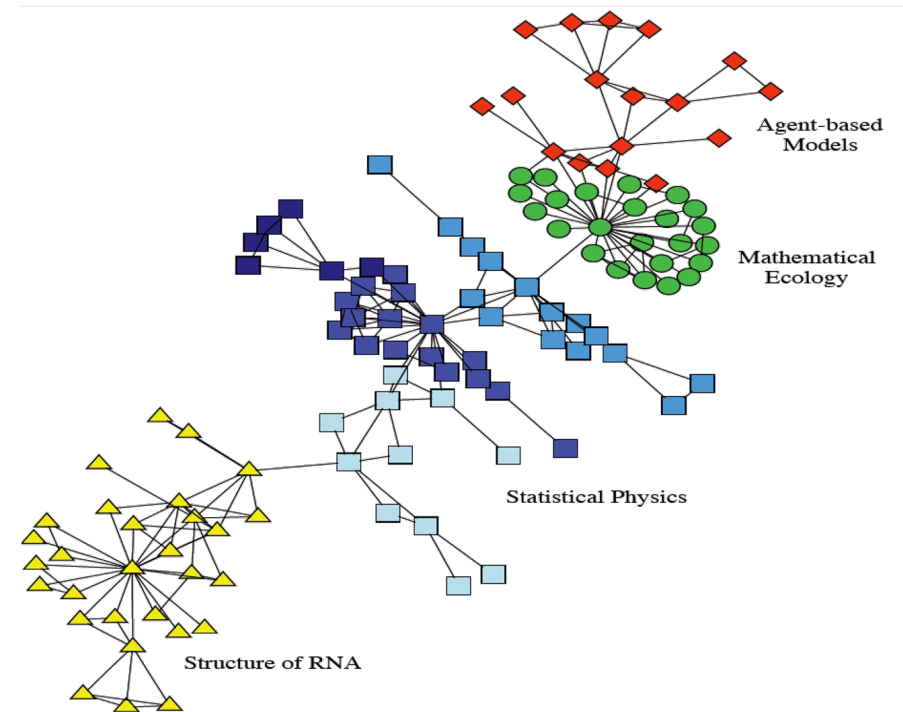
- Aquí cada ejemplo completo ya es un grafo (probablemente pequeño)
- Ejemplos:
 - clasificar una molécula como tóxica o no.
 - clasificar una red como normal o anómala.
 - clasificar un conjunto de transacciones como fraudulentos o no.

Clasificación, regresión y *clustering* de grafos

- Se puede considerar un tipo de tarea distinta. La inferencia se hace sobre el grafo completo (no sobre componentes individuales como vértices o aristas).
- El *dataset* para entrenamiento es un **conjunto de grafos**, y el objetivo es hacer **predicciones independientes para cada grafo**.
- Es la tarea más análoga al aprendizaje supervisado estándar (no supervisado si hacemos *clustering*): cada grafo es un caso de entrenamiento independiente (se cumple la condición i.i.d.).
- El desafío es encontrar los atributos que tengan en cuenta la **relación estructural** dentro de cada caso de entrenamiento.

Detección de comunidades

- Sirve para explorar estructura, aunque también para inferir propiedades de ciertos nodos.
- Normalmente es aprendizaje no supervisado
- Ejemplos:
 - grupos de usuarios muy conectados.
 - subgrupos funcionales.
 - *clusters* de comportamiento.



Clustering y detección de comunidades

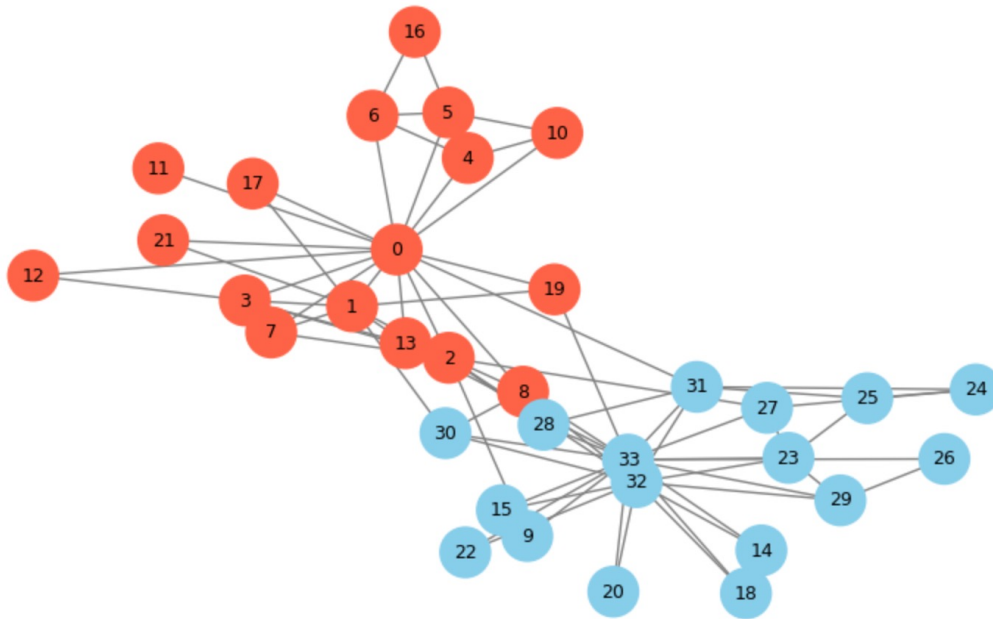
- Tanto la clasificación de nodos como la predicción de relaciones implican inferir información desconocida sobre los datos del grafo, y en gran medida son el equivalente al aprendizaje supervisado.
- La detección de comunidades es en cambio más análoga al aprendizaje no supervisado.
- Objetivo: inferir las estructuras de comunidades latentes a partir solo del grafo de entrada $G = (V, A)$.



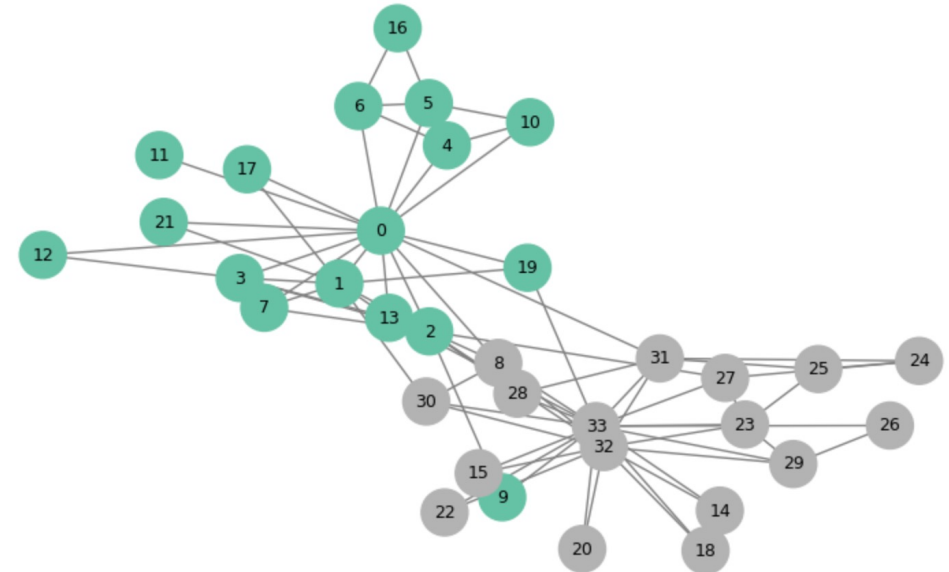
Ver ítems 1 - 2

Comparación del resultado del algoritmo

Karate Club coloreado por grupo real



Comunidades detectadas con greedy_modularity_communities



Vamos a pensar

- ¿Debo recomendar esta película a este usuario?
 - Netflix, Spotify, ...
- ¿Este usuario es spam (cuenta falsa)?
- ¿Qué grupos naturales hay aquí?
- ¿Dos usuarios se harán amigos?
- ¿Esta molécula será activa?



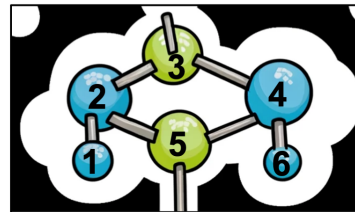
La vida antes de las GNN (Redes Neuronales basada en Grafos)



- Antes de las GNN, lo habitual era extraer **características del grafo** o **generar embeddings** y luego usar **modelos clásicos**.

Opción 1: convertir el grafo en variables

- Idea:
 - calculamos **métricas** de cada vértice.
 - luego usamos esas métricas como **features** (atributos) .
- Ejemplos de *features*:
 - Grado
 - PageRank
 - Otras medidas de centralidad
 - Clustering coefficient*.



Molecular graph
of six atoms

	f1	f2	f3	f4
1				
2				
3				
4				
5				
6				

Node feature
matrix

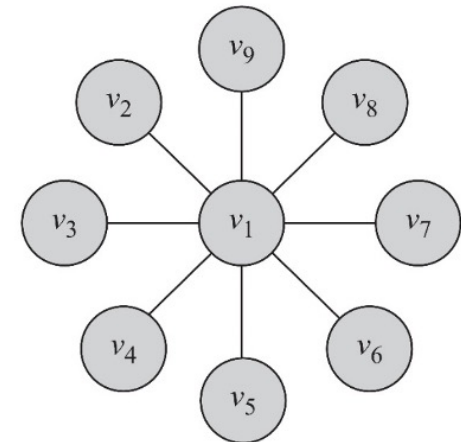
	1	2	3	4	5	6
1	1	1	0	0	0	0
2	1	1	1	0	1	0
3	0	1	1	1	0	0
4	0	0	1	1	1	1
5	0	1	0	1	1	0
6	0	0	0	1	0	1

Adjacency
matrix

Ejemplos de métricas: centralidad de grado

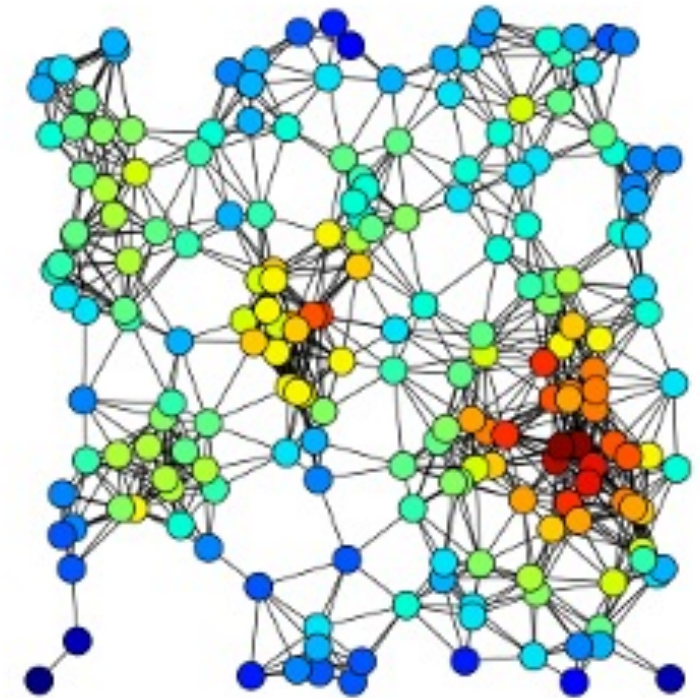
- **El grado de un nodo** es la **cantidad de conexiones** que tiene.
 - En X (Twitter), número de seguidores.
 - En Facebook, número de amigos.
 - Se denota d_i .
- **Centralidad de grado**: grado del nodo **normalizado**.
 - Se denota $C_D(i)$
 - Por ejemplo, por el grado máximo $C_d^{\max}(v_i) = \frac{d_i}{\max_j d_j}$

$$C_D(1) = \frac{8}{8} = 1$$



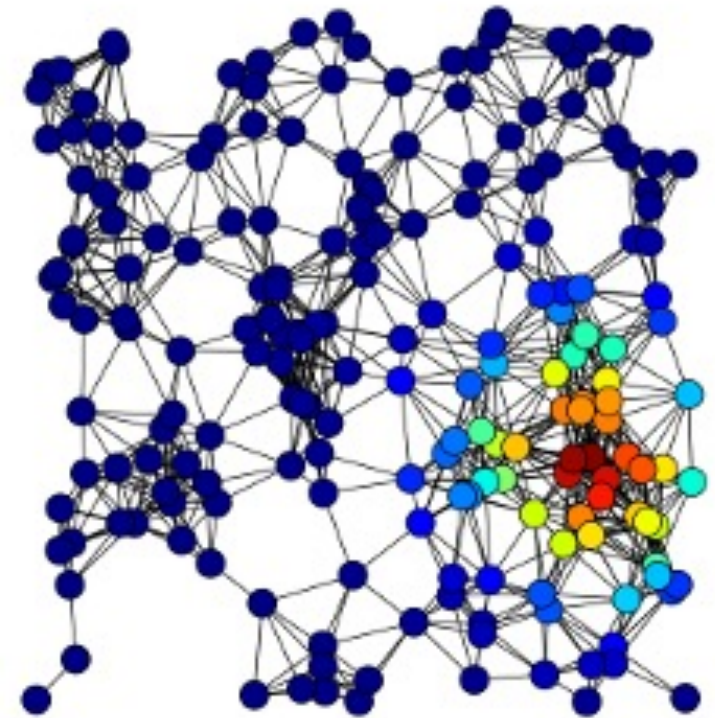
Ejemplos de métricas: centralidad de grado

- Es la métrica **más simple**, pero ofrece información útil sobre la importancia de los nodos.
- Orientada a encontrar las “**celebridades**” de la red social
 - Suele haber **pocos**, y son más populares que el resto en **varios órdenes de magnitud**.



Ejemplos de métricas: centralidad de PageRank

- Versión recursiva de la centralidad de grado.
- En vez de sumar 1 por cada arco, se le da un peso equivalente al grado del nodo con el que conecta.
- Un nodo es central en la medida que esté conectado con otros nodos centrales.
 - Conectado con muchos nodos con muchas conexiones.



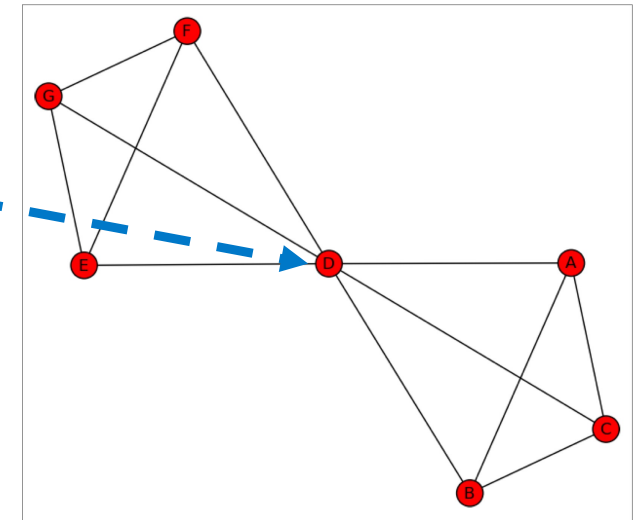
Ejemplos de métricas: centralidad de Intermediación

Tipo de nodo que busca destacar

¿Cuán importante eres conectando a otros?

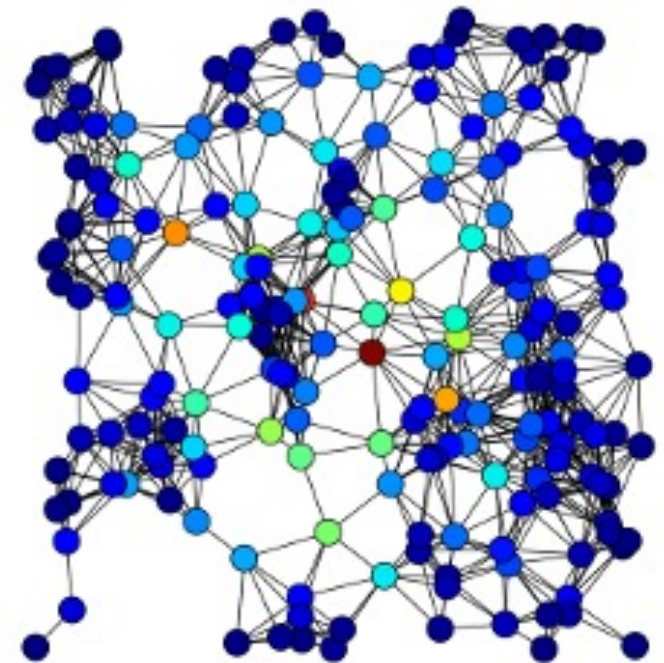
Presunción: los actores importantes son los que tienen **control** sobre otros.

- Si 2 actores j, k , **no adyacentes** interactúan, y un actor i está en el camino entre j y k , entonces i puede tener cierto **control** sobre las interacciones de j y k .
- Si i está en el camino de muchas de estas interacciones, debe ser un actor **importante**.



Ejemplos de métricas: centralidad de Intermediación

- Busca los **puentes** o **cuello de botella**.
- El nodo central no necesariamente tiene muchas conexiones, sino que es un punto de conexión entre muchos pares.
- **Costoso de calcular.**



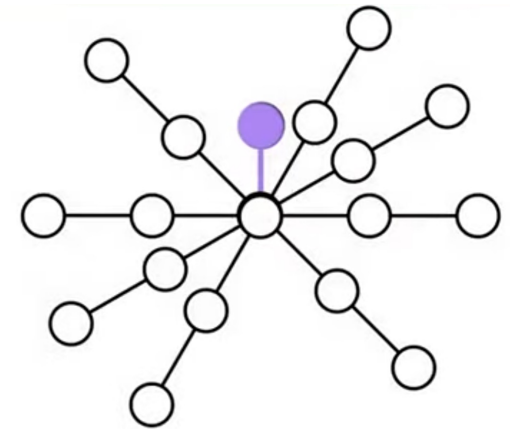
Ejemplos de métricas: centralidad de Cercanía

- La cercanía puede definir el rol de una persona en la red

- Distancias (lejanía): $\sum_y d(y, x)$

- La inversa es cercanía $C_{(x)} = \frac{1}{\sum_y d(y, x)}$

- Normalizando por tamaño de la red $C_{(x)} = \frac{N}{\sum_y d(y, x)}$



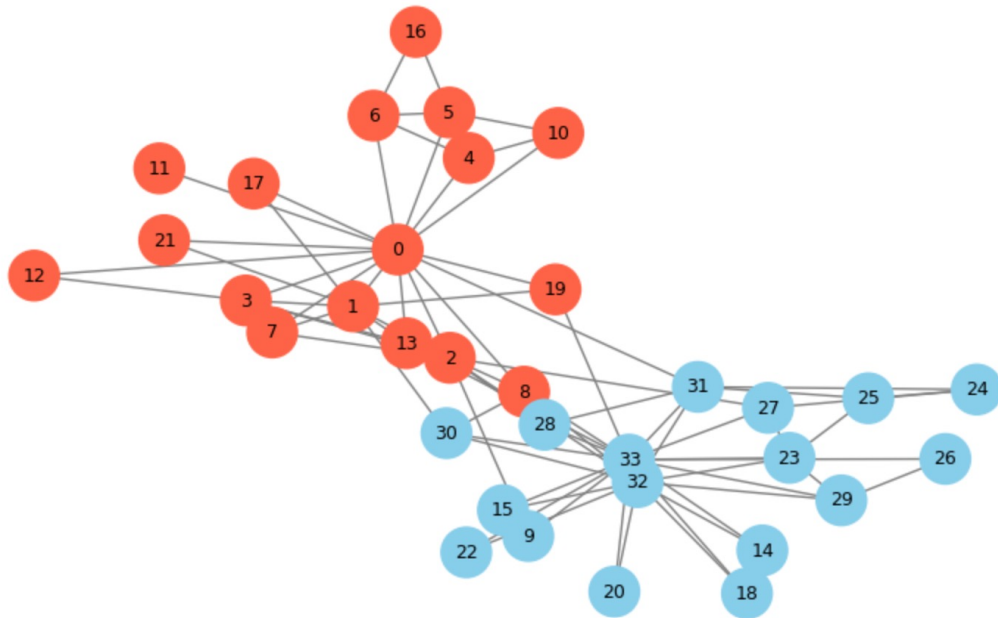
Ver ítem 3

Ejemplo de pipeline clásico

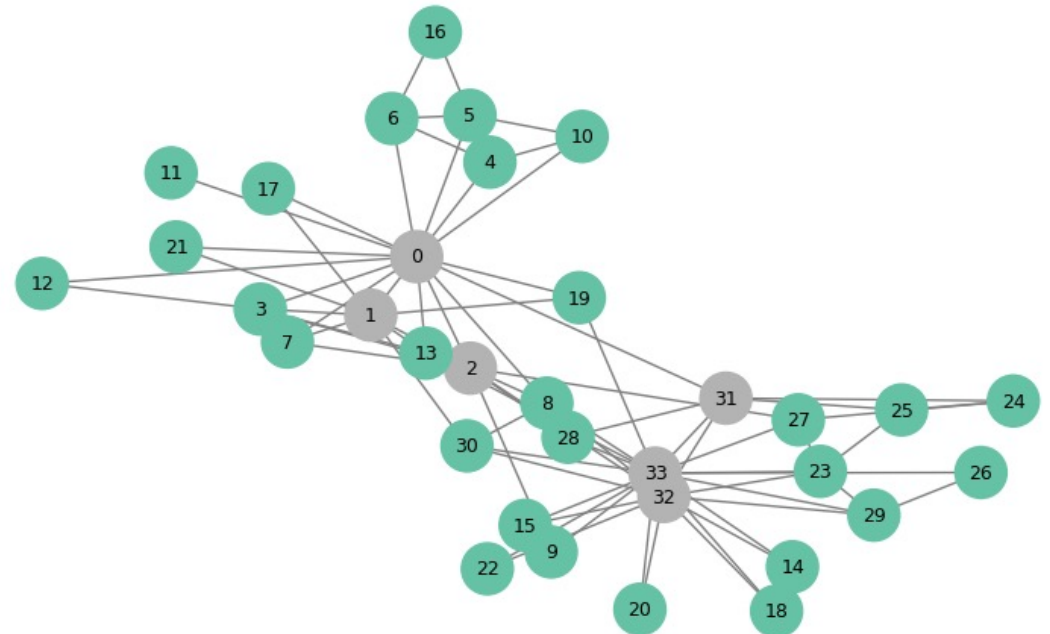
- Grafo → métricas por vértice → dataset tabular → clasificador
- Características:
 - Enfoque muy común.
 - Simple y útil.
 - Pero requiere diseñar variables a mano.

Comparando la inferencia con la realidad

Karate Club coloreado por grupo real



Nodos agrupados según sus medidas de centralidad



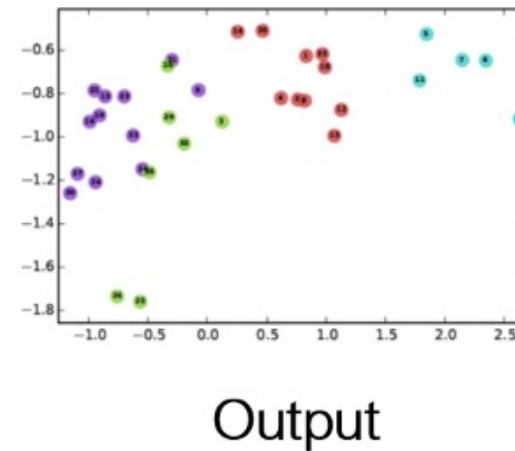
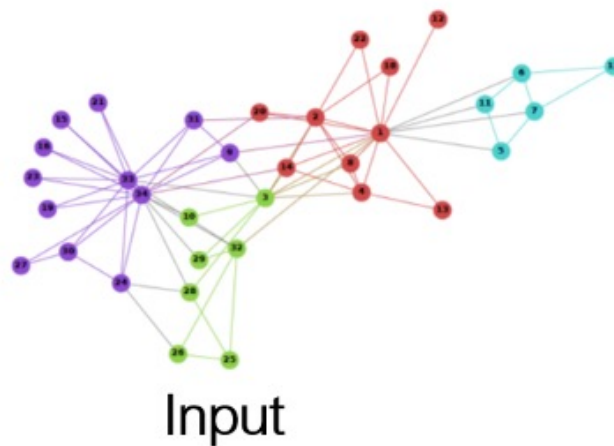
Opción 2: *embeddings* de vértices

- Intuición:

- Cada vértice se representa como un **vector** (de bajas dimensiones).
- Vértices **parecidos** o **cercanos** deberían tener **representaciones parecidas**.

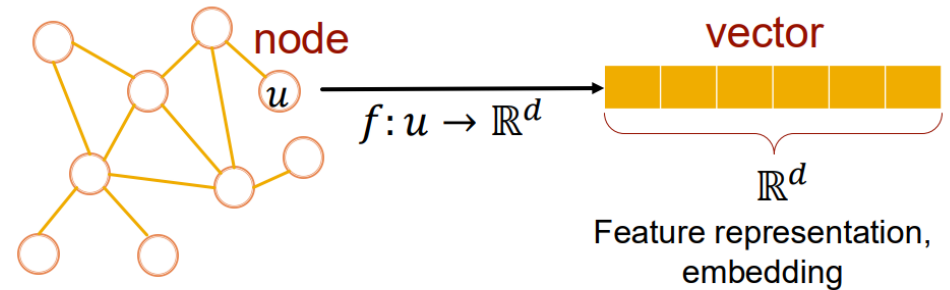


Ver ítem 4



Opción 2: *embeddings* de vértices

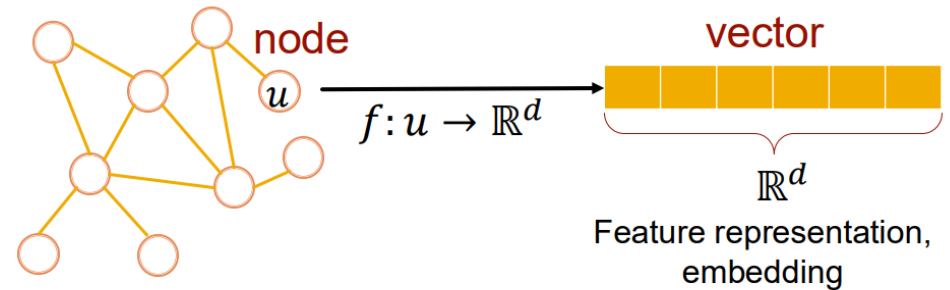
- Vértice / arista / Grafo $\rightarrow$ *Embedding*
- Similitud de vértices se traduce en cercanía de *embeddings*
 - ¿Qué es similitud?
 - ¿Qué es cercanía?



Opción 2: *embeddings* de vértices

- Vértice / arista / Grafo $\rightarrow$ *Embedding*
- Similitud de vértices se traduce en cercanía de *embeddings*
 - ¿Qué es similitud?
 - ¿Qué es cercanía?

¡DEPENDENDE!



Similitud y cercanía

- Cercanía de *embeddings*:
 - Distancia euclídea, distancia coseno, distancia de Mahalanobis, ...
- Similitud:
 - Depende del problema / algoritmo

Opción 2: Node2Vec

- Técnica inspirada en Word2Vec
- Objetivo: crear *embeddings* (vectores de bajas dimensiones) para los nodos de un grafo.

Cercanía == Distancia coseno

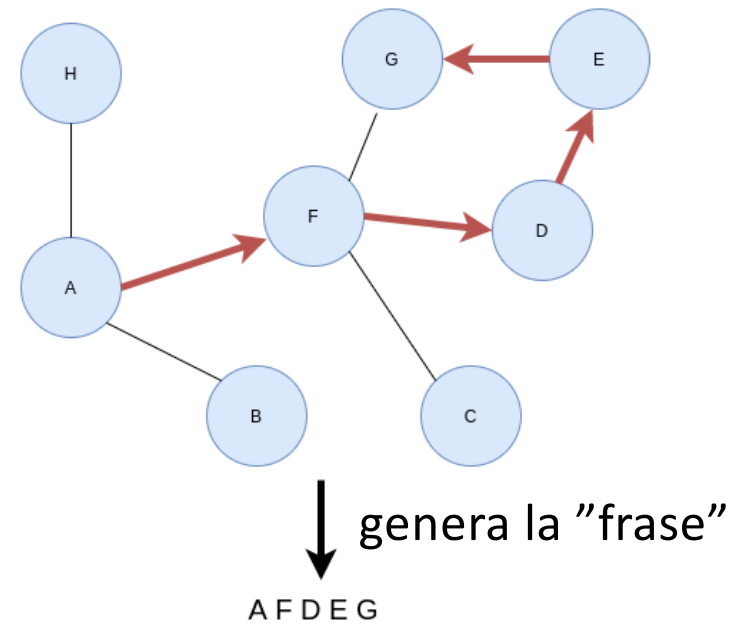
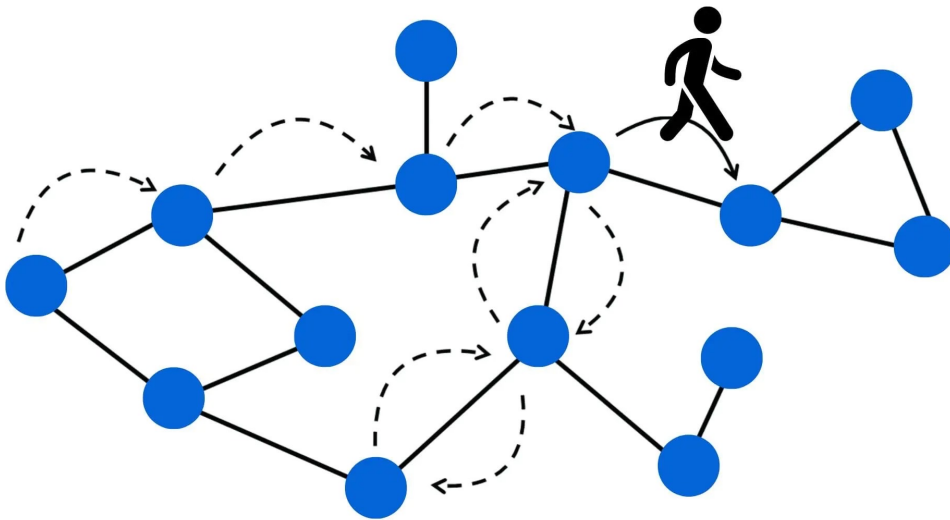
A blue curved arrow points from the text 'Cercanía == Distancia coseno' to the equation $\mathbf{z}_u^T \mathbf{z}_v \approx$. Another blue curved arrow points from the text 'Similitud == Co-ocurrencias en caminos aleatorios' to the same equation. The text 'probability that u and v co-occur on a random walk over the graph' is written in blue and is part of the equation's right-hand side.

$$\mathbf{z}_u^T \mathbf{z}_v \approx \text{probability that } u \text{ and } v \text{ co-occur on a random walk over the graph}$$

Similitud == Co-ocurrencias en caminos aleatorios

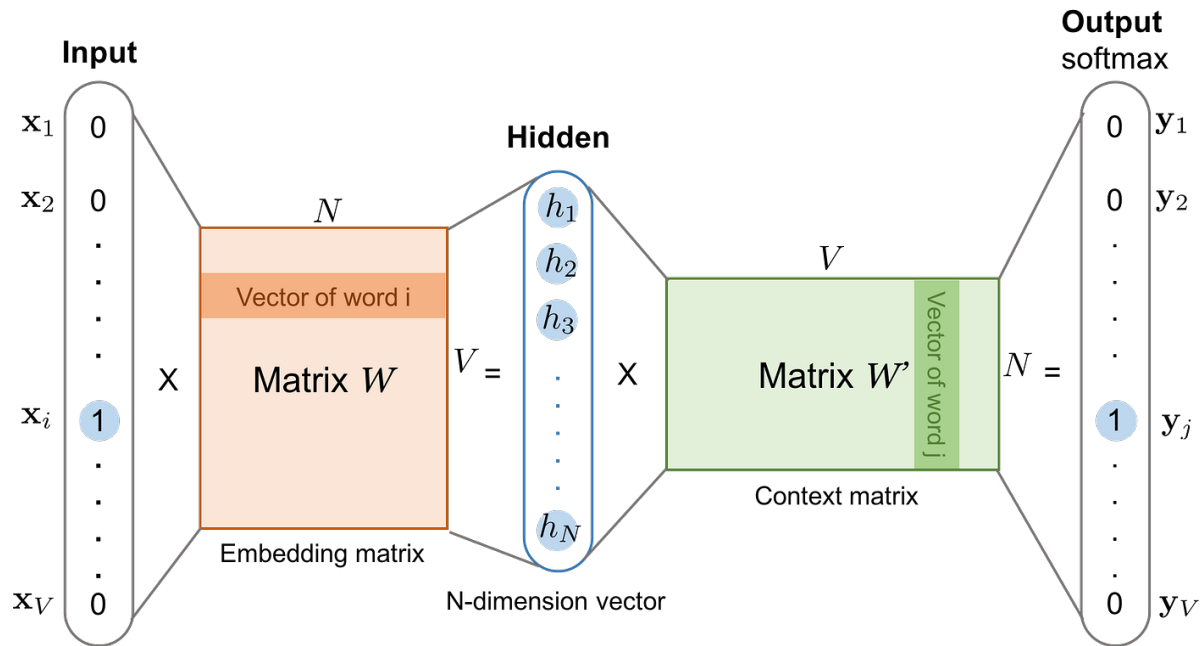
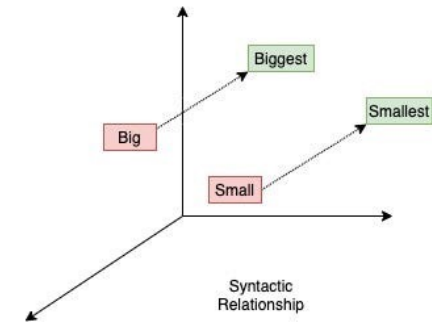
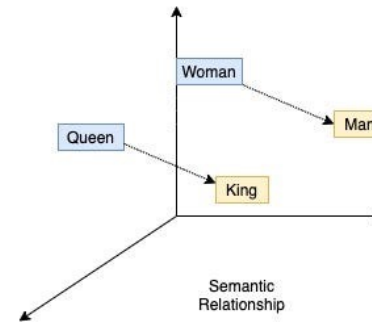
Opción 2: *embeddings* de vértices

- Inspirados en soluciones del ámbito de NLP.
 - Hacen recorridos por la red y aprenden representaciones como si los nodos fueran “palabras” de un lenguaje estructural



Opción 2: *embeddings* de vértices

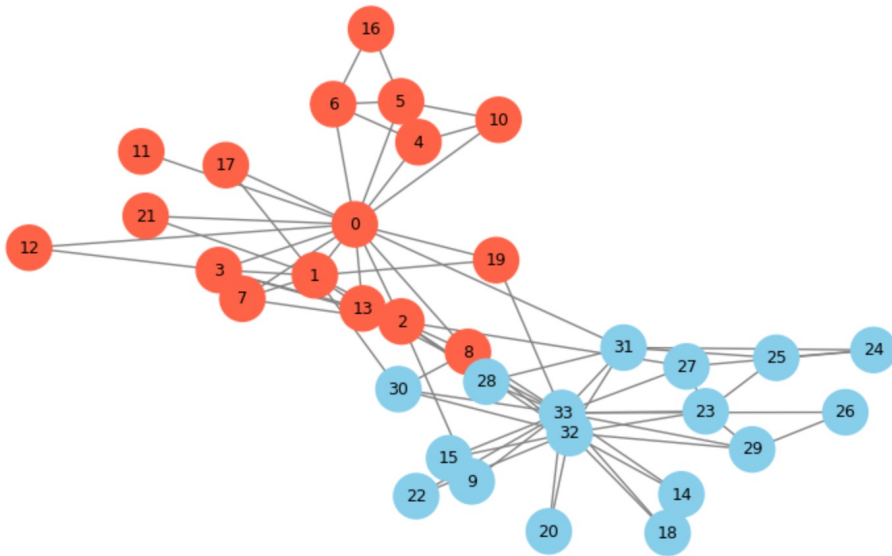
- Métodos comunes:
 - Node2Vec (coje ideas del Word2Vec)
 - DeepWalk



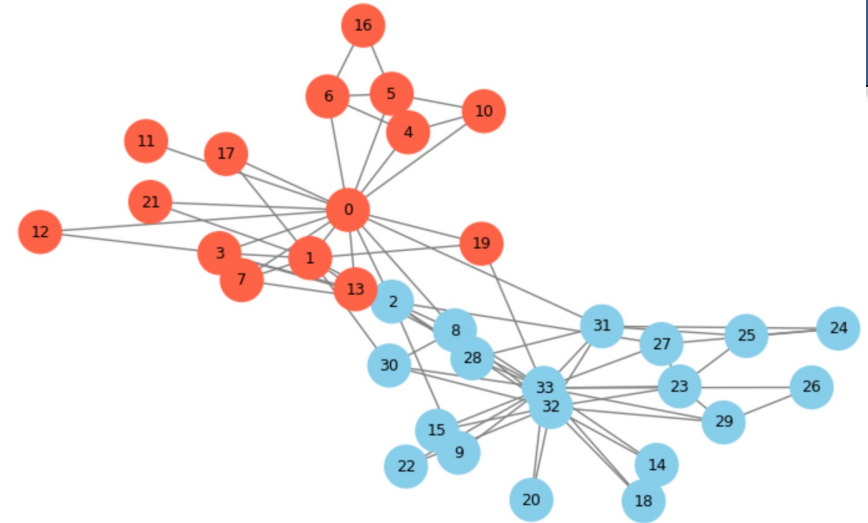
Ver ítem 5

Aprendizaje Automático basado en Grafos

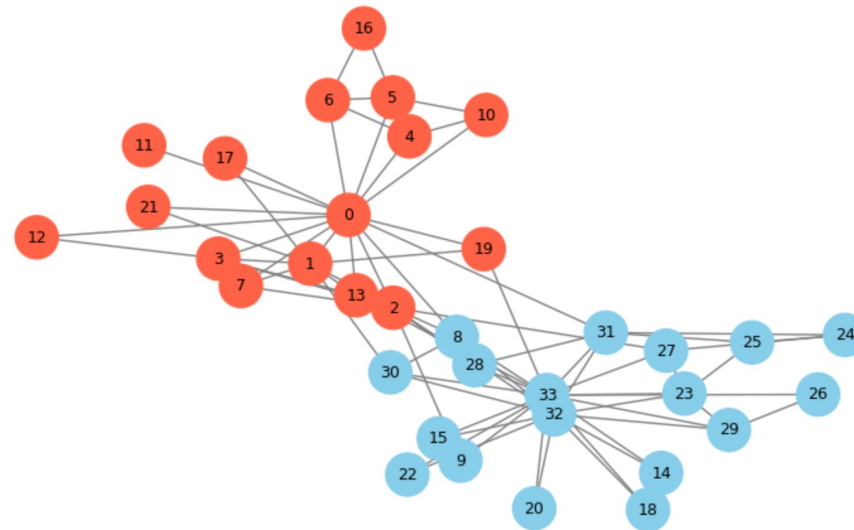
Karate Club coloreado por grupo real



Karate Club coloreado por grupo usando DeepWalk + Clustering



Karate Club coloreado por grupo usando DeepWalk + NN



Qué tenían de bueno estos enfoques

- Funcionan razonablemente bien
- Son más simples de entender
- Se pueden combinar con modelos clásicos
- En muchos casos siguen siendo competitivos
- **Sin embargo:**
 - Mucha ingeniería manual.
 - Cuesta **integrar** atributos y estructura de forma conjunta.
 - La tarea final no siempre guía la representación.
 - No siempre capturan bien el contexto multinivel.
 - Dependen de muestras aleatorias

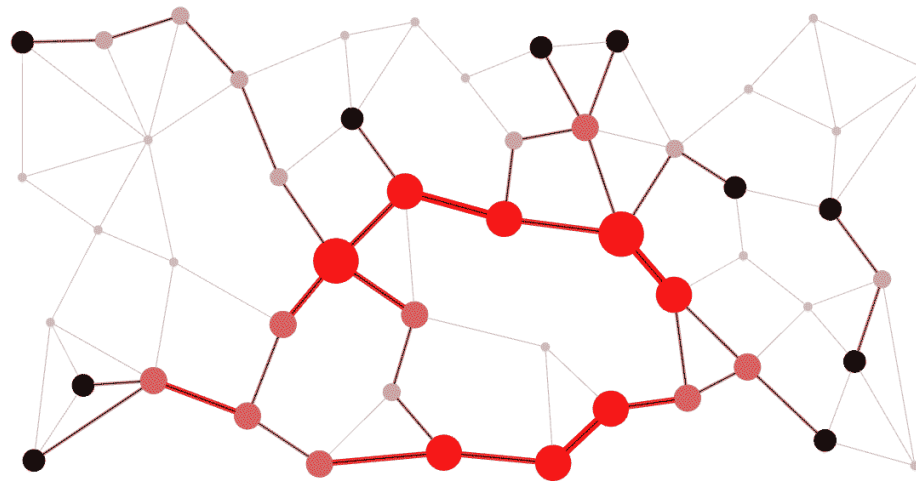
Caminando hacia GNN

- ¿Qué ventaja tiene convertir un vértice en un vector?
- ¿Qué problema veis en diseñar a mano las características de todos los nodos?
- ¿Puede pasar que una métrica simple como el grado sea insuficiente? ¿Por qué?
- ¿Y si el propio modelo pudiera aprender a combinar automáticamente la información de cada nodo con la de sus vecinos?



¿Qué es una GNN?

- Una *Graph Neural Network* es un modelo que aprende sobre grafos actualizando la representación de cada nodo a partir de su propia información y la de sus vecinos.



Idea intuitiva de *message passing* (pasaje de mensajes)

- Cuenta con 3 pasos:
 - cada nodo tiene información inicial
 - recibe información de sus vecinos
 - actualiza su representación.

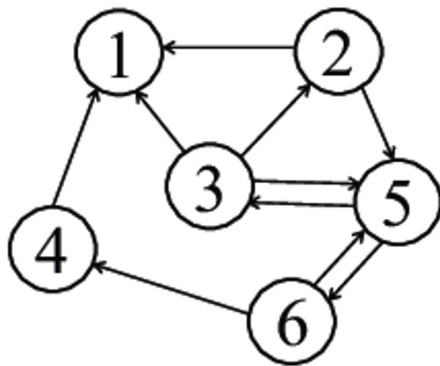


Analogía social

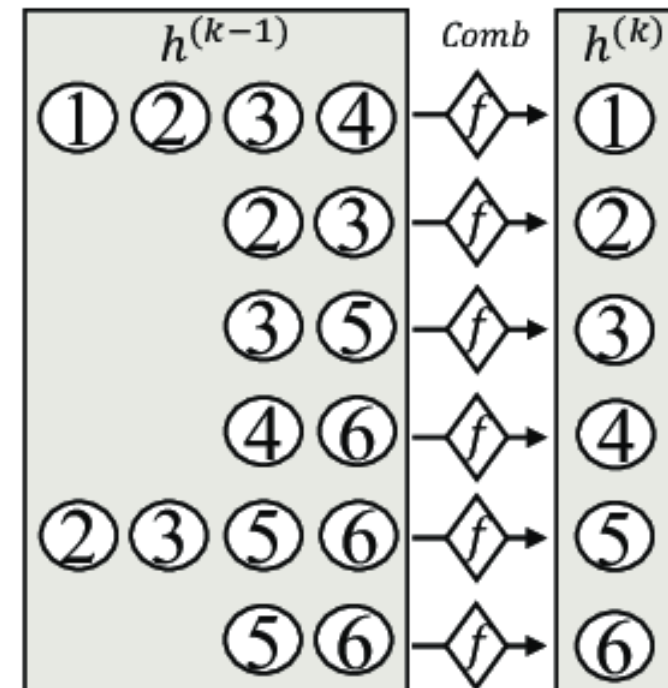
- Ejemplo:
 - una persona puede parecer “normal” por sus atributos
 - pero si está rodeada de cuentas claramente sospechosas, eso añade señal de alerta
- El **contexto importa**.

Con una sola capa

- Nodo 1
- Vecinos 2, 3 y 4
- 1 recoge sus señales y las resume

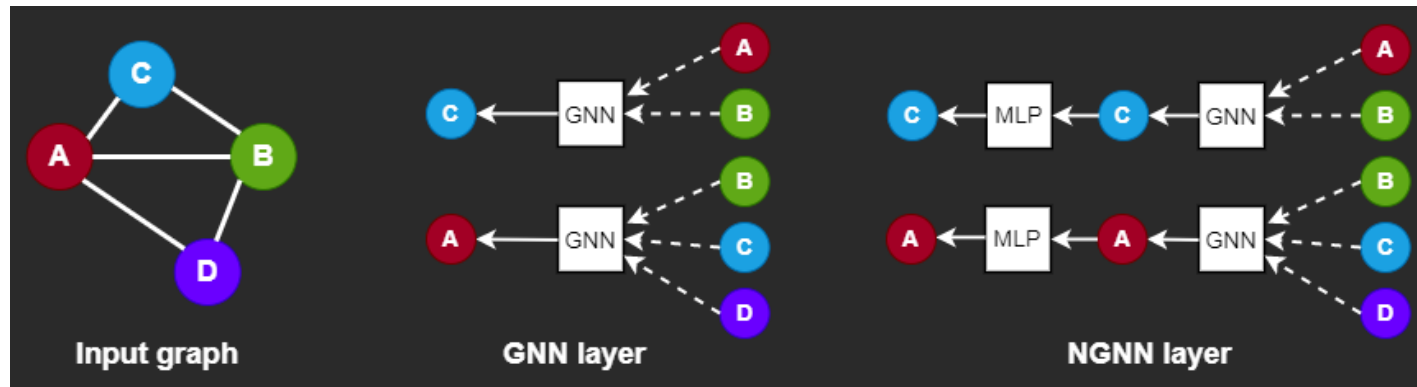


Entrenamiento
de 1 capa



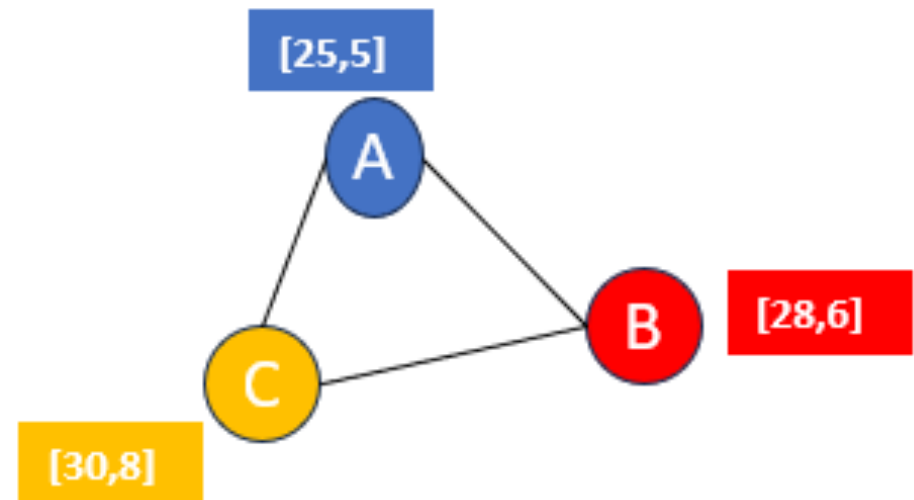
Con varias capas

- Con 1 capa: conoce a sus vecinos directos.
 - Con 2 capas: conoce vecinos de vecinos.
 - Con [3..] capas: llega más lejos en la red.
-
- Más capas = más contexto.
 - Pero también pueden traer problemas (*oversmoothing*).



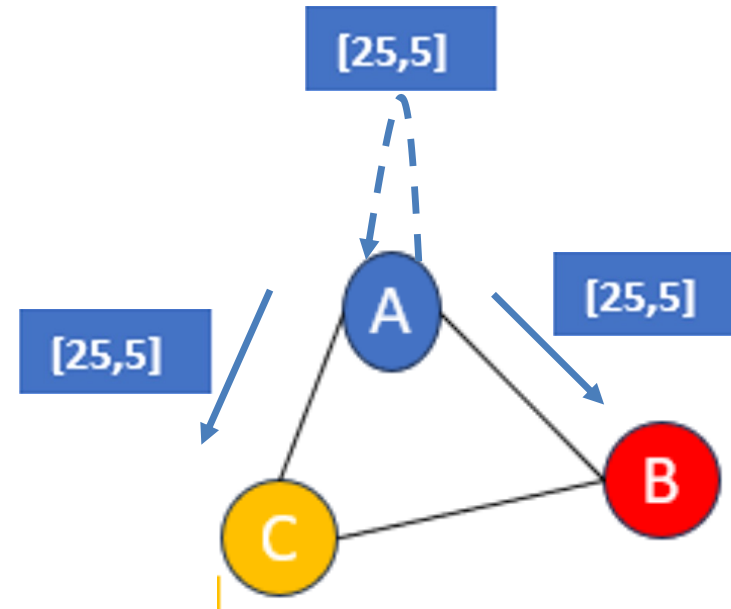
Esquema visual : estado inicial

- A=Alice; B=Bob; C=Carol
- $[X,Y] = [\text{Edad}, \text{Intereses}]$



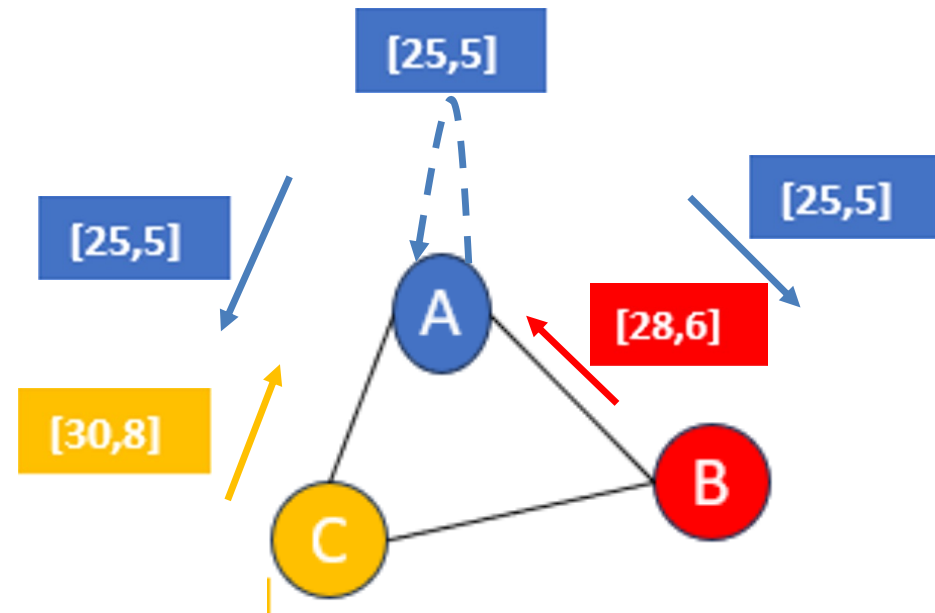
Esquema visual : Mensaje

- Alice $\rightarrow$ Bob; [25,5]
- Alice $\rightarrow$ Carol; [25,5]
- (Alice $\rightarrow$ Alice; [25,5])
- Bob $\rightarrow$ Alice; [28, 6]
- ...
- Carol $\rightarrow$ Bob; [30,8]
- (Carol $\rightarrow$ Carol; [30, 8])



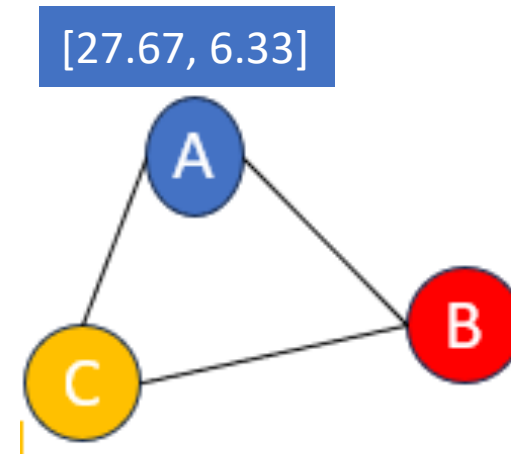
Esquema visual : Agregación

- Alice
 - $([25,5] + [28,6] + [30,8]) / 3$
 - Ejemplo de función de agregación

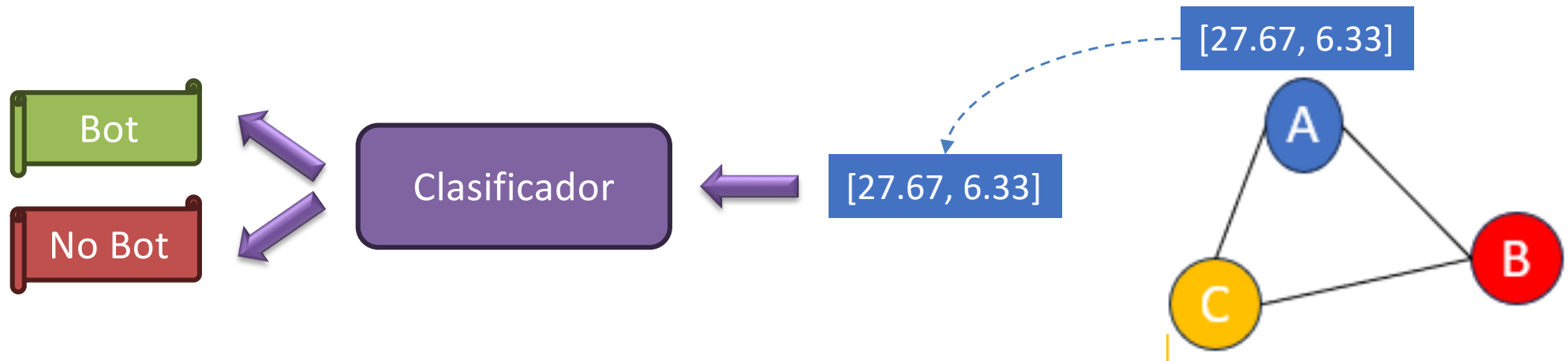


Esquema visual : Actualización

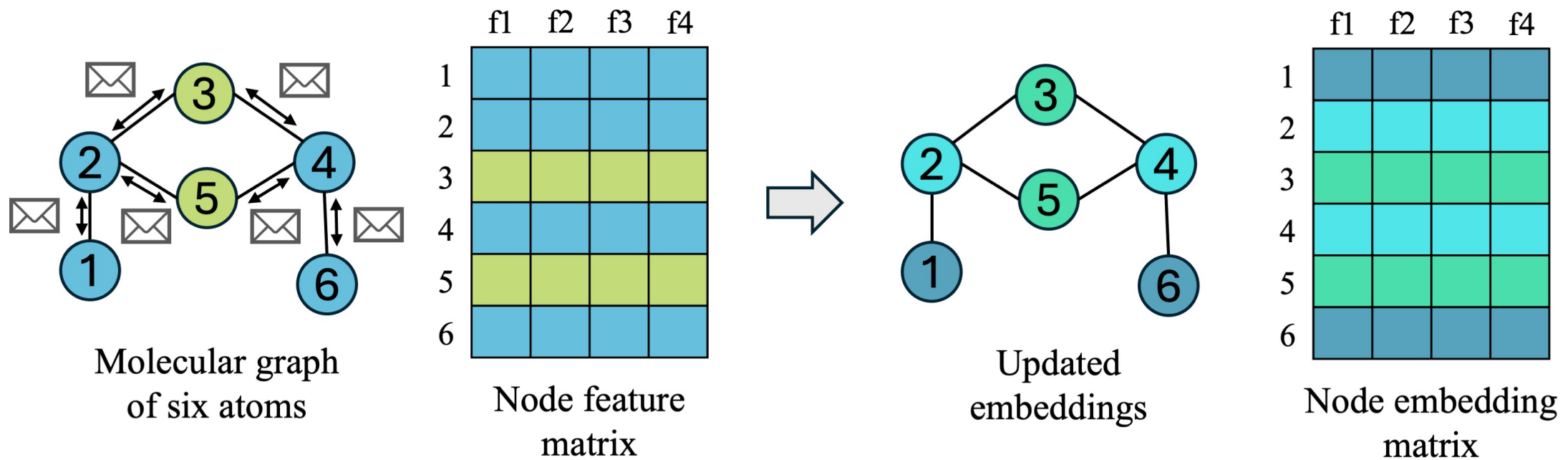
- Alice
 - $([25,5] + [28,6] + [30,8]) / 3 = [27.67, 6.33]$
 - Ejemplo de función de agregación



Esquema visual : Clasificación



Resultado del aprendizaje



Similitudes con Deep Learning

- Aprenden representaciones
- Tienen capas
- Se entrenan para una tarea
- Minimizan una función de pérdida

Diferencias con CNN e imágenes

- En imágenes la estructura es regular. En grafos la estructura es irregular
 - Cada nodo puede tener distinto número de vecinos
- Esta es una de las dificultades.
- Tres familias:
 - **GCN**: mezcla información local de vecinos
 - **GAT**: aprende que no todos los vecinos importan igual
 - **GraphSAGE**: útil para grafos grandes, muestrea vecinos

Tres familias

- GCN:
 - promedia o combina lo que llega del vecindario.
 - buena primera intuición de convolución en grafos.
- GAT:
 - unos vecinos son más relevantes que otros.
 - el modelo aprende pesos de atención.
- GraphSAGE:
 - Si el grafo es enorme, no puedes usar siempre todos los vecinos
 - Tomas una muestra y agregas sobre ella.

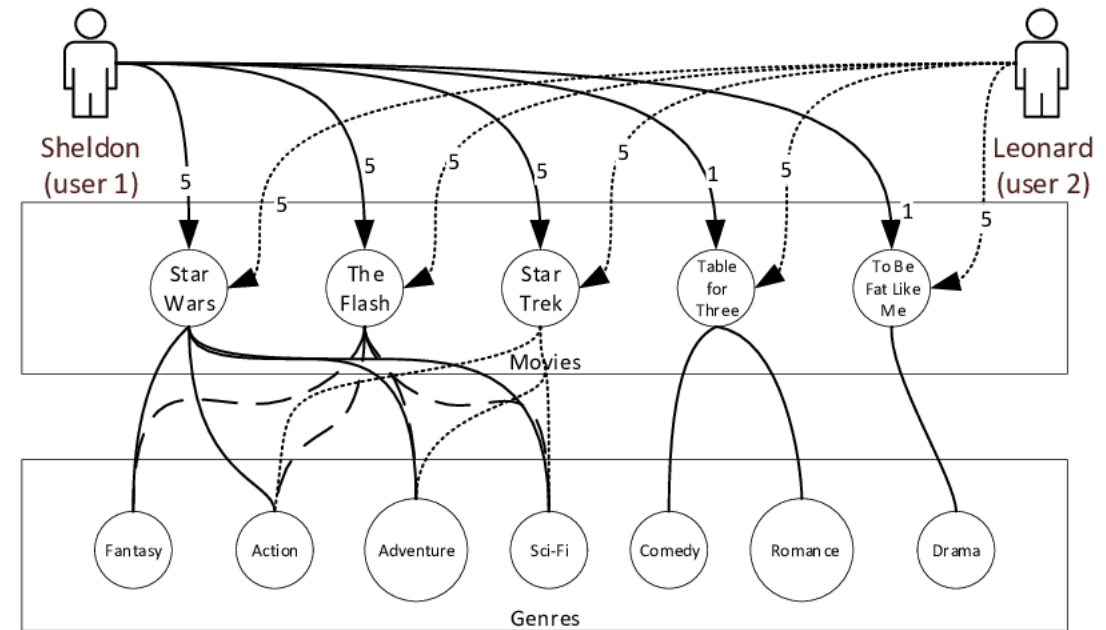
A tener en cuenta

- GNN no significa “meter un grafo y mágicamente resolver todo”
- Antes hay que decidir:
 - qué son vértices
 - qué son aristas
 - qué atributos tenemos
 - cuál es la tarea exacta
- ¿Por qué puede ser útil que un nodo “mire” a sus vecinos?
- ¿Qué gana un nodo con dos capas frente a una?
- ¿Qué problema podría haber si seguimos metiendo capas y capas?
- ¿Creéis que todos los vecinos deberían influir igual?



Recomendación

- Grafo bipartito usuario–item
- Idea:
 - nodos de dos tipos
 - interacciones como aristas
 - se puede predecir qué enlaces faltan o cuáles serán relevantes



Fraude y ciberseguridad

- Nodos:
 - cuentas bancarias
 - dispositivos
 - tarjetas
 - Ips
- Se busca:
 - conexiones anómalas
- una cuenta aislada puede parecer normal
- dentro de una red sospechosa deja de parecerlo

Redes Sociales

- Tareas:
 - recomendación de amistad.
 - detección de bots.
 - detección de comunidades.
 - difusión de información.

Moléculas y química

- átomo = nodo
- enlace = arista
- el grafo completo representa una molécula
- Tareas:
 - toxicidad
 - actividad biológica
 - propiedades químicas

¿Cómo resumo todo el grafo en un vector?

- No se debe caer en la tentación de aumentar la profundidad para abarcar todo el grafo.
 - *Oversmoothing*: Los nodos se vuelven demasiado parecidos entre sí, el *embedding* del grafo pierde riqueza.
 - *Oversquashing*: demasiada información lejana intenta comprimirse en representaciones de tamaño limitado; información importante puede quedar aplastada.
 - Entrenamiento más difícil: más capas implica más coste, más inestabilidad.
- Normalmente: *message passing* de pocas capas + *global pooling*.
 - Ejemplos de *global pooling*:
 - Media.
 - Suma.
 - Atención / *pooling* aprendido (el modelo aprende qué nodos pesan más en la representación global).

Grafos de conocimiento / documentos / citas

- documentos conectados por citas
- entidades conectadas por relaciones semánticas
- ayuda en clasificación, recuperación y razonamiento

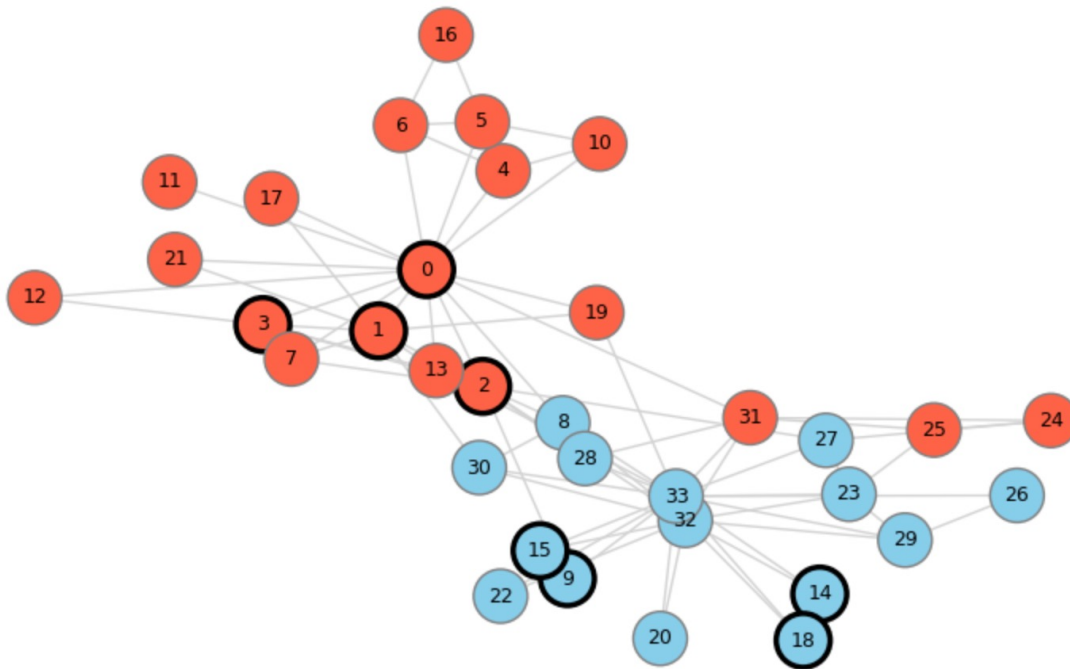
Preguntas



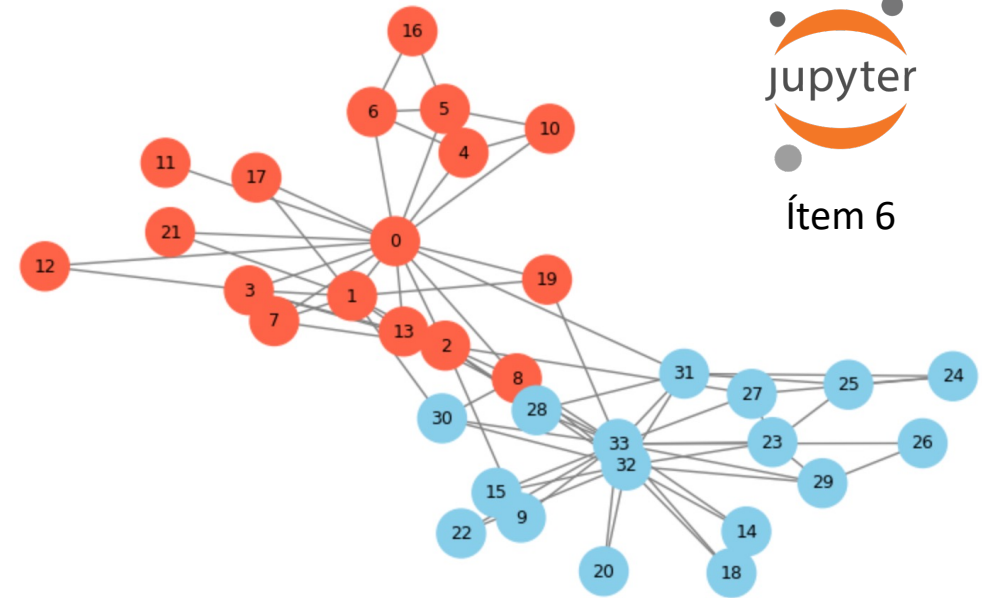
- ¿Qué caso os parece más intuitivo: ¿redes sociales, recomendación o moléculas?
- ¿Dónde creéis que el contexto relacional aporta más valor que las variables individuales?
- ¿Se os ocurre un problema de vuestro entorno que pueda modelarse como grafo?

Ejemplo (pequeño) de funcionamiento

Predicciones de la GCN
(Borde negro = nodos etiquetados para entrenamiento)

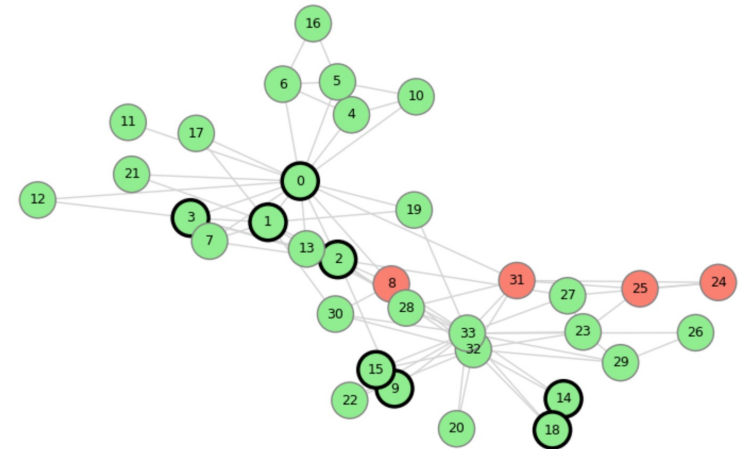


Karate Club coloreado por grupo real



Ítem 6

Aciertos y errores de la GCN
(verde = correcto, rojo = error)

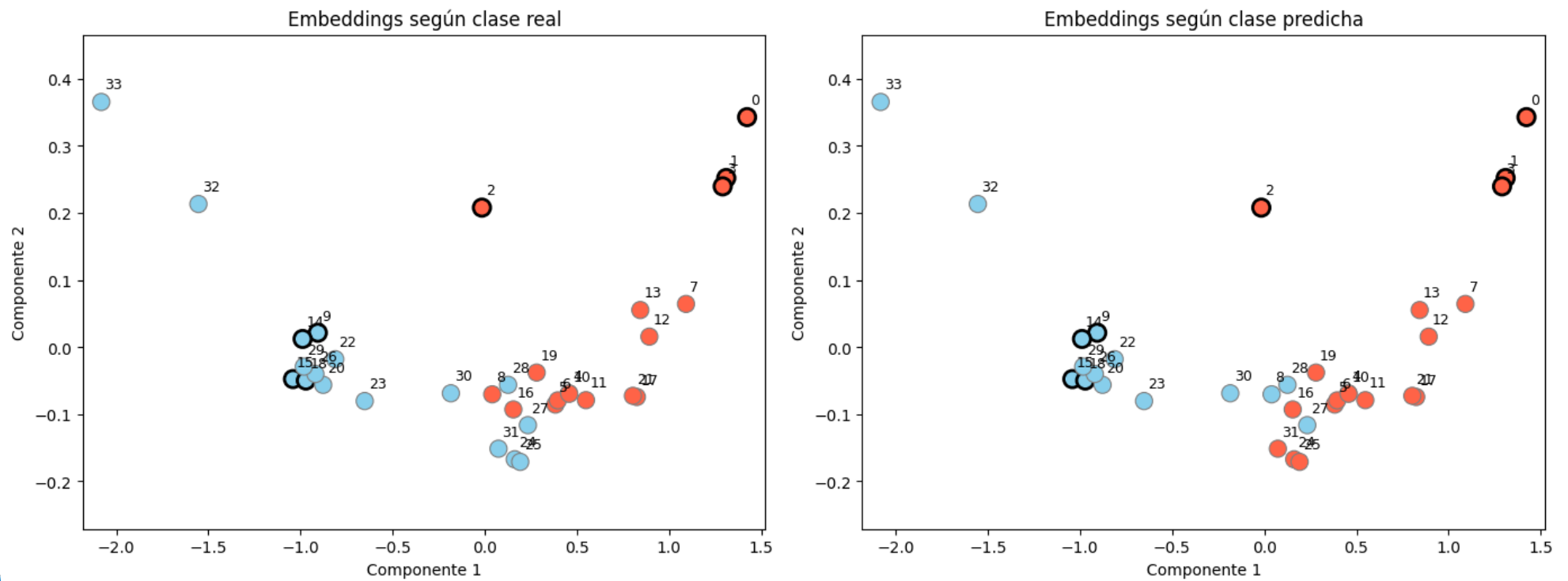


Ejemplo (pequeño) de funcionamiento



Ítem 7

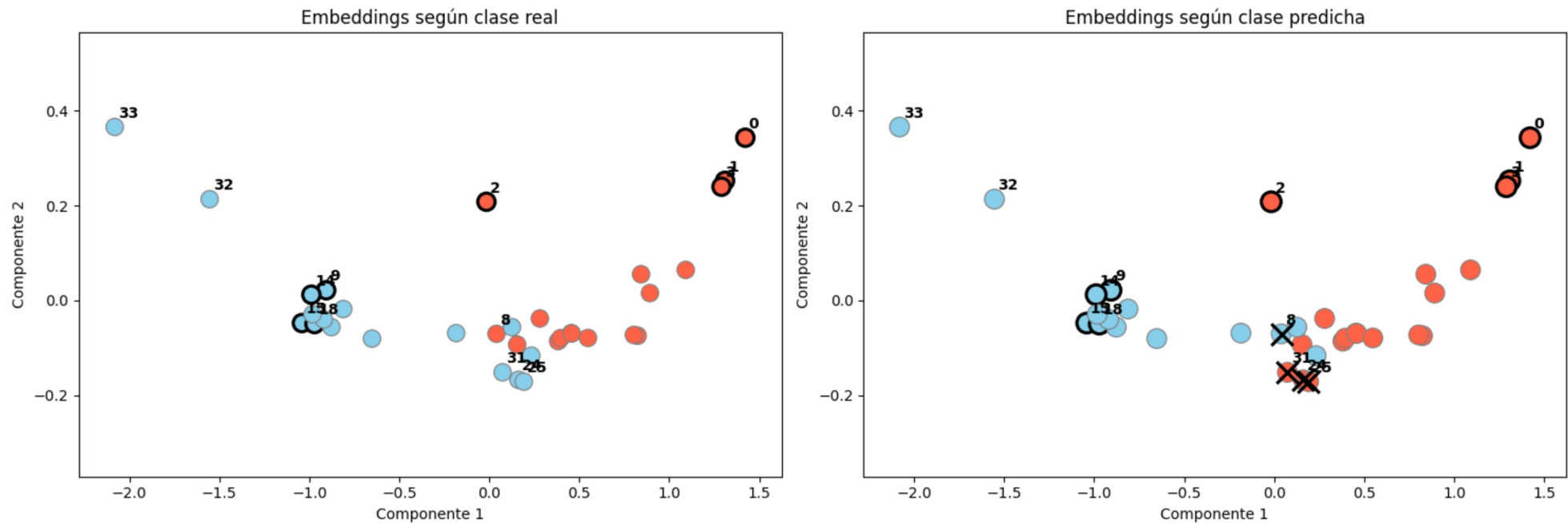
Comparación de embeddings: clase real vs clase predicha



Ejemplo (pequeño) de funcionamiento

● Mr. Hi ● Officer ○ Nodo de entrenamiento ✕ Error de clasificación Etiquetas: nodos destacados

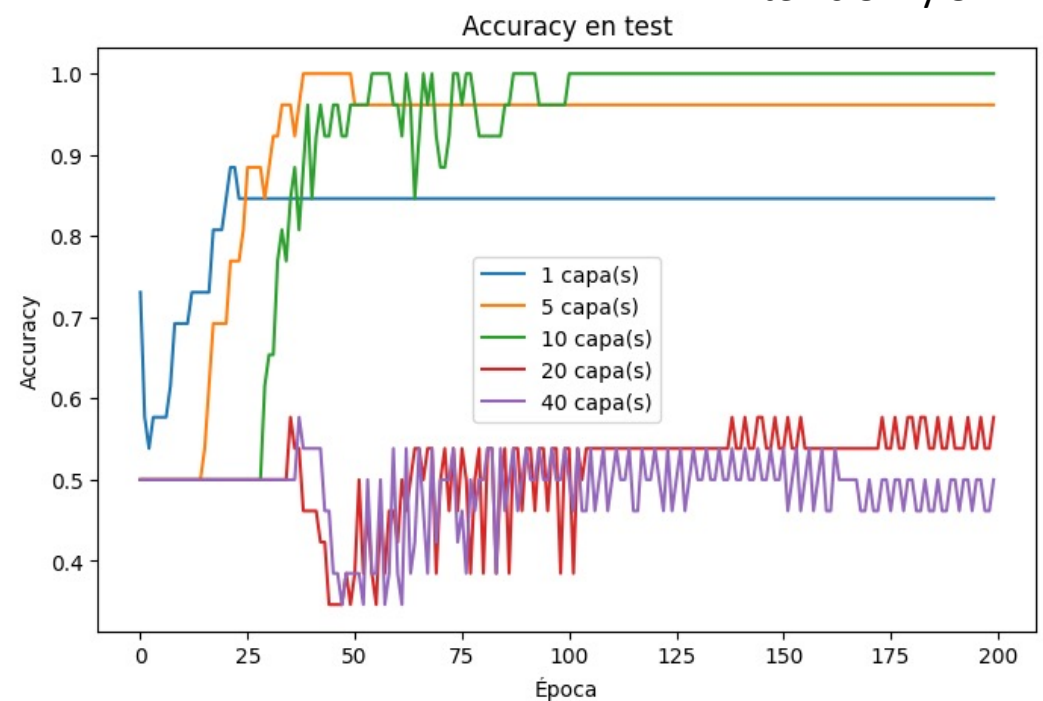
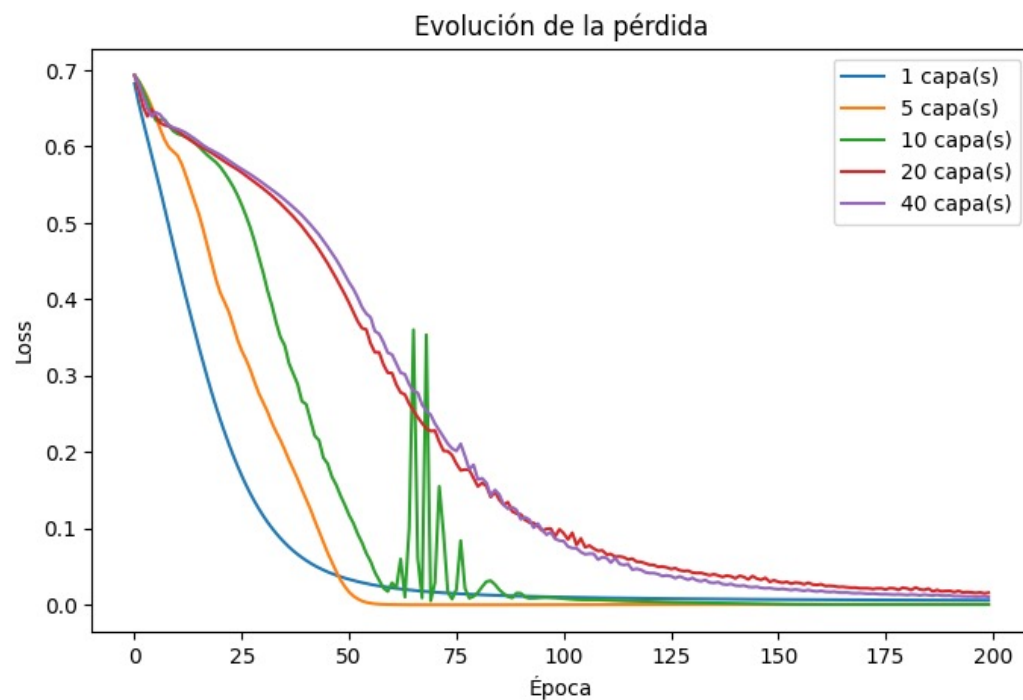
Comparación de embeddings: clase real vs clase predicha



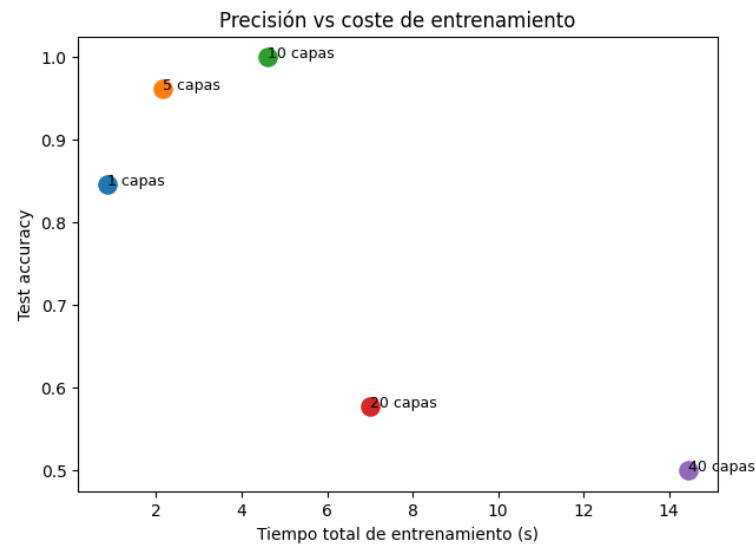
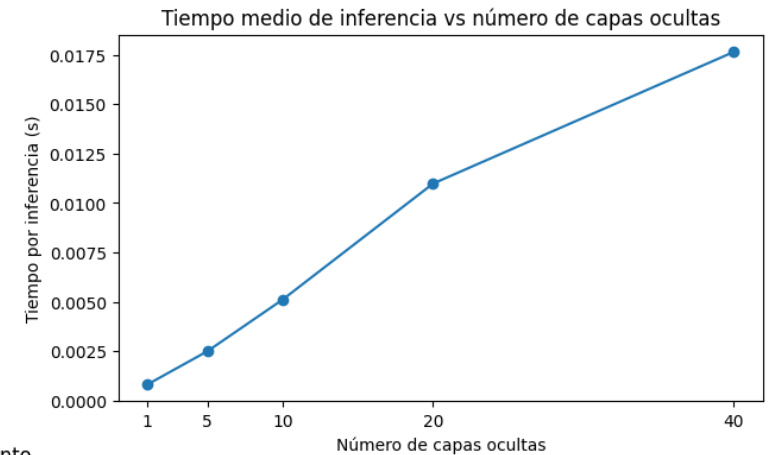
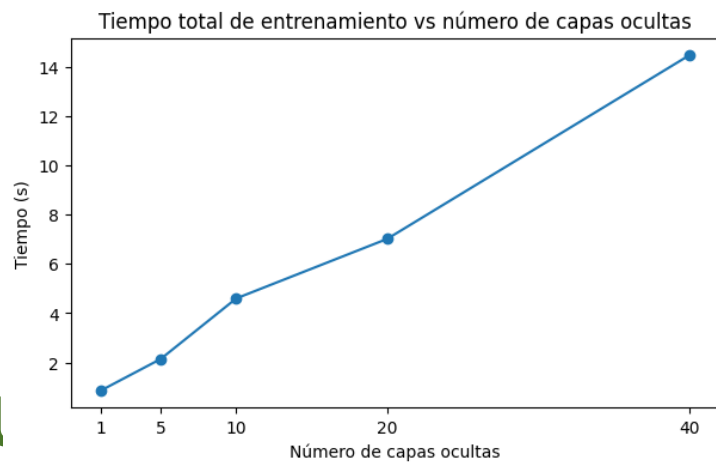
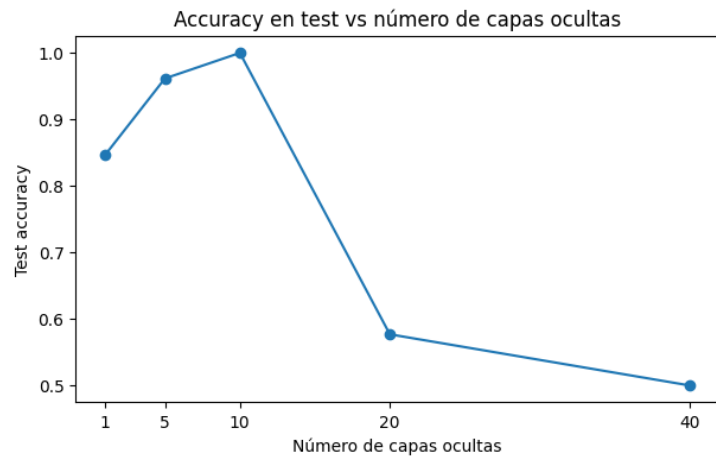
Comparación usando diversos números de capas



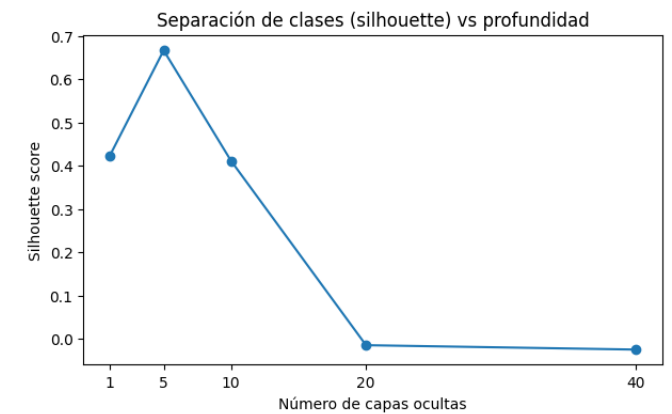
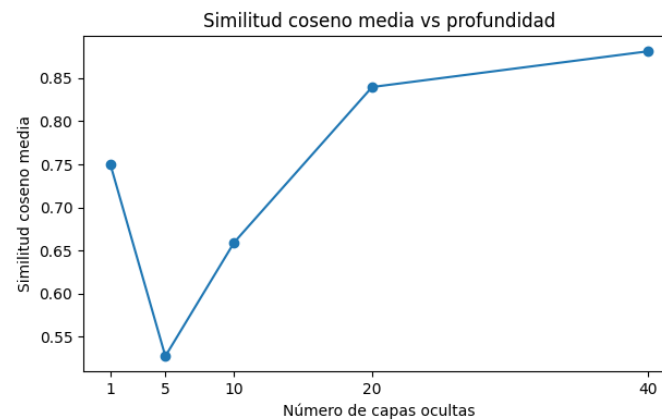
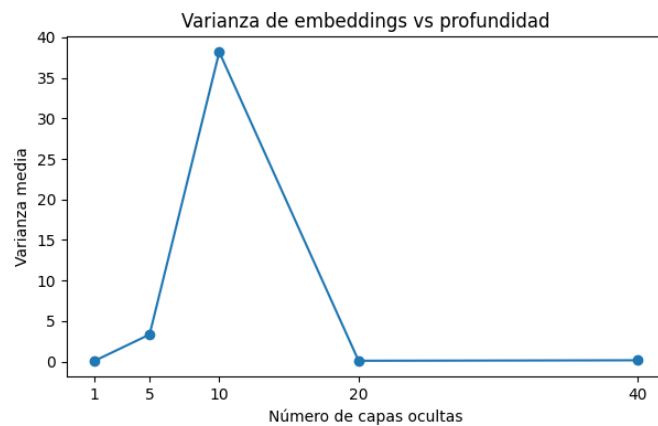
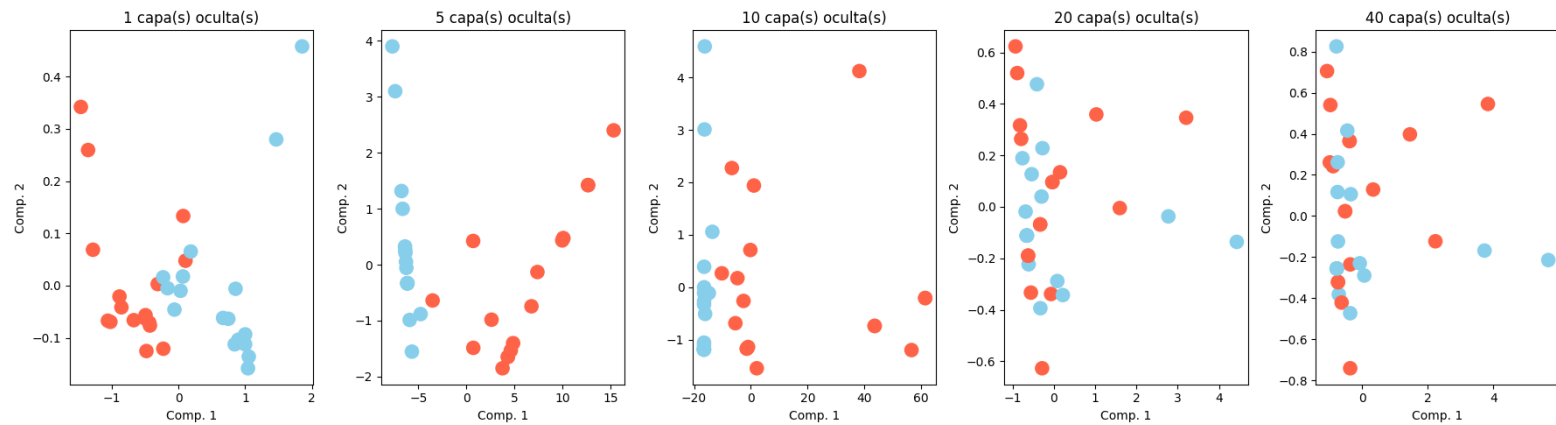
Ítems 8.1 y 8.2



Comparación usando diversos números de capas



Comparación de *embeddings*



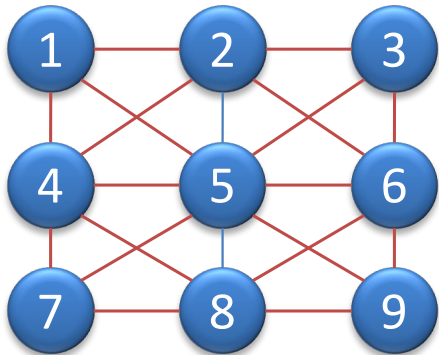
Limitaciones y problemas reales

- Construir el grafo correcto no es trivial
- No siempre sabemos qué significa una arista
- Escalabilidad en grafos grandes
- Entrenamiento más costoso
- Interpretabilidad limitada
- Sesgos estructurales (sumados a los sesgos de las propiedades de los individuos)
- Oversmoothing (homogeneización) en modelos profundos

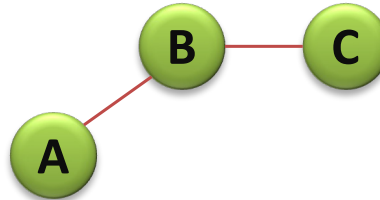


Sesgos estructurales

Grupo A



Grupo B



Consecuencia

Predicciones más
fiables en grupo A

Predicciones peores o
más inestables en
grupo B

Cuándo merece la pena usar grafos

- Sí:
 - si las relaciones son parte del problema
 - si hay dependencia entre entidades
 - si la estructura aporta señal
- No necesariamente:
 - si los ejemplos son independientes
 - si forzar un grafo solo complica sin aportar valor

Las 5 ideas que debéis llevaros a casa

1. No todos los datos son tablas
2. Los grafos representan entidades y relaciones
3. Se puede aprender sobre nodos, enlaces o grafos completos
4. Las GNN combinan información propia y vecinal
5. Usar grafos tiene sentido solo cuando las relaciones importan de verdad

Idea final

En muchos problemas de IA,
entender a una entidad exige
entender también su posición en
una red.



Escuela Politécnica Superior

DS y ML

Machine Learning y Deep Learning

Graph Machine Learning

Aprendizaje Automático basado en Grafos

Álvaro Ortigosa, alvaro.ortigosa@uam.es

